



Bahir Dar University
Bahir Dar Institute of Technology Faculty of Computing
Requirement Specification Document (RAD & RDD)
for
Integrated Digital Payment And Fare Management System
For Shared Urban Transport

Submitted to the faculty of computing in partial fulfillment of requirements for the degree of
Bachelor of Science in Software Engineering

Group Members

Name	ID Number
1 BetreMariam Yihenew	1404554
2 Nigist Abayneh	1404958
3 Surafel Muhabaw	1404539

Advisor: Demeke Endalie

Bahir Dar University, Bahir Dar Institute of Technology (2018)

Declaration

The Project is our own and has not been presented for a degree in any other university and all the sources of material used for the project have been duly acknowledged.

Betremariam Yihenew



Name

Signature

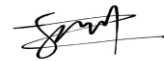
Nigist Abayneh



Name

Signature

Surafel Muhabaw



Name

Signature

Faculty: Computing

Program: Software Engineer

Project Title: Integrated Digital Payment and Fare Management System For Shared Urban Transport

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

Demeke Endalie



Name of Advisor

Signature

Examining committee members

signature

Date

1. Examiner1 _____

2. Examiner2 _____

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Acknowledgment

We would like to express our sincere and profound gratitude to Bahir Dar Institute of Technology, Faculty of Computing, for providing us with the invaluable opportunity to undertake this industrial project. The knowledge, resources, and academic environment offered by the institution played a significant role in the successful completion of this work.

We extend our special appreciation to our project advisor, Mr. Demeke E., whose continuous guidance, constructive feedback, and professional expertise were instrumental in shaping the quality and direction of this Requirement Analysis Document. His dedication and mentorship greatly enhanced our understanding and ensured the project adhered to academic and industry standards.

We are also grateful to the Chapa API support team for granting access to comprehensive technical documentation and support resources, which were essential for integrating and understanding the payment service components of the system.

Finally, we would like to express our heartfelt thanks to our families and group members for their unwavering support, encouragement, and effective collaboration throughout the duration of this project. Their commitment, patience, and teamwork were vital to overcoming challenges and achieving the project objectives.

Bahir Dar Institute of Technology

December 2018

Roles and Responsibilities Of The Group Memebers

This matrix details the assignment of tasks to each group member for the project, ensuring a clear division of labor and accountability for all deliverables.

List of Tasks	Betremariam	Nigist	Surafel
RAD Documentation Tasks			
Draft Chapter 1: Introduction (Background, Problem, Objectives)			√
Define Functional and Non-Functional Requirements		√	
Document Business Rules and Constraints (e.g., 3% commission, manual payout)	√		
System Modeling Tasks			

Develop Use Case Diagrams and Documentation			√
Design Activity Diagrams (Fare Payment, Top-up, Payout)		√	
Design Sequence Diagrams (Fare Payment, Login, Top-up)	√		
Design the Unified Analysis Class Model (Class Diagram)			√
Implementation Tasks (Technology Stack)		√	
Backend API Development (Node.js/Express/Prisma)	√		

List of Tasks	Betremariam	Nigist	Surafel
Database Schema Design and Implementation (PostgreSQL)	√		
Web Admin Portal Development (React.js/Zustand)			√
Mobile Application Development (Flutter/Riverpod)		√	
Specific Feature Implementation		√	
QR Code Generation and Validation Logic			√
Balance Management and Atomic Transaction Logic	√		
Chapa Payment Gateway Integration (Top-ups)	√	√	
Excel Report Generation for Payouts	√		
Testing and Quality Assurance			√
Unit and Integration Testing	√	√	√
System Testing and User Acceptance Testing (UAT)		√	√

List of acronyms:

F_AUTH_1	Function Authorization
F_PASS	Function Passenger
F_DRIV	Function Driver
F_AGNT	Function Agent
F_SADM	Function Sub Admin(Admin)
F_ADM	Function Admin
F_WAL	Function Wallet
F_NOTIF	Function Notification
F_REP	Function Report
BR	Business Role
NFR_P	Non Functional Requirement Perfomance
NFR_S	Non Functional Requirement Security
NFR_R	Non Functional Requiement Reliability
NFR_M	Non Functional Requirment Maintaiinability
CC	Change Case

List Of Tables

Table 1 Stake Holder	27
Table 2 Authentication User Management.....	34
Table 3 Passenger Requirement	35
Table 4 Driver Requirement.....	35
Table 5 Agent Requirment	36
Table 6 Admin Super Admin	38
Table 7 Buisness Role Documentation.....	45
Table 8 Non functional Requirment.....	59
Table 9 Hardware Requirment.....	59
Table 10 Software Requirment	60
Table 11 Change Case	62
Table 12 Access Control Matrix	91
Table 13 Test Summary.....	105
Table 14 Field Observation Notes.....	107
Table 15Chapa vs Telebirr.....	108

List of Figures

Figure 1 Use Case Diagram	39
Figure 2 Detailed Use Case Diagram.....	40
Figure 3 Passenger Mobile Application.....	46
Figure 4 Passenger Mobile Application.....	47
Figure 5 Prototype 3: Web Admin Portal (Super Admin)	47
Figure 6 Passenger Balance LifeCycle	48
Figure 7 Agent Commission Balance Status	49
Figure 8 Driver Earning Status	50
Figure 9 Activity Diagram For Fare Payment process.....	51
Figure 10 Activity Diagram - Agent Top-up and Commission Process	52
Figure 11 Analysis Class Model	53
Figure 12 Sequence Diagram For Fare Payment Interaction.....	54
Figure 13 Sequence Diagram Agent Top Up Transaction	55
Figure 14 Logic Model For Agent Commission Calculation.....	56
Figure 15 Logic Model Driver Earning Reset	57
Figure 16 Class Diagram	61
Figure 17 User Interface Prototype.....	63
Figure 18 Architectural Design.....	67
Figure 19 Component Modeling	68
Figure 20 Deployment Modeling.....	70
Figure 21 Design Class Model.....	71
Figure 22 Persistence Class Model	72
Figure 23 User Interface-Landing Page	73
Figure 24 User Interface - wallet Top-up.....	74
Figure 25 User Interface - Passenger and Driver Dashboards	75
Figure 26 User Interface - Agent Dashboard	75
Figure 27 User Interface - Transaction History	76
Figure 28 Web Admin - Driver Payment Management	77
Figure 29 Web Admin - Agent Commission Management	77
Figure 30 Web Admin - Analytics Dashboard.....	78
Figure 31 Web Admin - QR Re-issue Requests	78
Figure 32 Web Admin - User Registration Form	79

Figure 33 Web Admin - User Management and Sub-admin Management	80
Figure 34 Web Admin - Fermata (Fare) Management.....	81
Figure 35 System Folder Structure	95
Figure 36 Authentication and Role-check Middleware Code.....	96
Figure 37 Transaction and Payment Processing Implementation Code.....	98
Figure 38 QR Code Management Implementation	99
Figure 39 Chapa Payment Integration Backend Implementation	101

Table of Contents

Declaration.....	1
Acknowledgment	3
Roles and Responsibilities Of The Group Memebbers	4
List of acronyms:	6
List Of Tables.....	7
List of Figures	8
Abstract.....	13
Chapter One: Introduction.....	15
1.1 Background	15
1.1.1 Current State of Shared Urban Transport Payments	15
1.1.2 Global Digital Transformation Trends and Local Relevance	15
1.1.3 The Proposed Hybrid QR-Based Solution	16
1.1.4 Stakeholder-Centric Design and Financial Inclusion.....	16
1.2 Statement of the Problem	17
Impact of These Problems on Transport Operations.....	17
1.3 Objectives.....	18
1.3.2 Specific Objectives	18
1.4 Methodology.....	19
1.4.1 Requirement Gathering Methods	19
1.4.2 Analysis and Design Methodology	20
1.4.3 Implementation Methodology	21
1.5 Feasibility	23
1.5.1 Economic Feasibility.....	23
1.5.2 Technical Feasibility	23
1.5.3 Time Feasibility	23
1.6.1 SWOT Analysis of the Proposed System	25
1.6.2 Stakeholder Analysis.....	26
1.7 Limitations of the Project.....	27
1.8 Scope of the Project.....	28
1.8.1 Functional Scope.....	29
1.8.2 Technical Scope	30
1.8.3 Exclusions	30

1.9. Organization of the Project.....	30
Chapter Two: System Features	32
2.1 The Existing System.....	32
2.2 Proposed Solution	32
2.3 Requirement Analysis	34
2.3.1 Functional Requirements	34
Passenger Module.....	34
Driver module	35
Agent Module	36
Admin / Super admin, wallet, and reporting	36
2.3.3 Business Rule Documentation	43
2.3.4 User interface prototype.....	45
User Interface Prototype Wireframe	46
2.3.5 State Chart Diagram.....	47
2.3.6 Activity Diagram.....	51
2.3.7 Analysis Class Model.....	53
2.3.8 Sequence Diagram	53
2.3.9 Logic Model.....	55
2.4 Non-Functional Requirements	58
2.5 System Requirments	59
2.5.1 Hardware Requirements.....	59
2.5.2 Software Requirements	59
2.6 Key Abstraction with CRC Analysis.....	60
2.6.1 Conceptual Modeling: Class Diagram	60
2.6.2 Identifying Change Cases	61
2.6.3 User Interface Prototype	62
Chapter 3: System Design	64
System Overview and Purpose	64
Design Goal: Performance	64
Design Goal: Dependability.....	65
Design Goal: End-User Experience and Accessibility	65
Design Goal: Scalability and Security	65
3.1 Architectural Design.....	66

3.1.1 Component Modeling	67
3.1.2 Deployment Modeling	68
3.2 Detailed Design	70
3.2.1 Design Class Model	71
3.2.2 Persistence Model – Introduction	72
3.3 User Interface Design.....	72
Wallet Top-Up User Interface	73
Dashboard Overview.....	74
Transaction Overview	76
Super Admin Payments Management Page	76
Super Admin Analytics Page	77
QR Code Management for Non-Smartphone Passengers	78
Agent and Driver Registration.....	79
User and Sub Admin Management.....	79
Fare Management Implementation (Admin Side)	80
3.4 Access Control and Security.....	81
Chapter Four: Implementation	94
System Architecture and Structure.....	94
Authentication and Authorization Implementation	95
Transaction and Payment Processing Implementation	96
QR Code Management Implementation.....	98
Passenger Wallet Initialization via Chapa	100
Summary	101
Chapter-Five: Testing and Evaluation	102
Testing Methodology	102
Authentication and User Management.....	102
Transaction Management.....	102
QR Code Operations	103
Administrative APIs.....	103
Super Admin Features.....	103
Sub-Admin Features	103
Mobile Application Testing	104
Passenger Features	104

Agent Features	104
Driver Features.....	104
Integration and Security Testing.....	105
Test Summary	105
References	106
Appendix:	107

Abstract

The Integrated Digital Payment and Fare Management System is a secure and efficient platform developed to modernize fare collection in shared urban transport services. The system aims to eliminate the challenges associated with cash-based transactions, such as delays, security risks, and lack of transparency, by introducing a digital payment solution based on QR code technology.

The platform is designed to be inclusive, supporting both smartphone users and non-smartphone users. Smartphone users access the system through a mobile application, while non-smartphone users utilize pre-registered printed QR codes, enabling them to participate in digital fare payments without owning a smartphone. QR codes are used for quick and secure fare validation, ensuring accurate transaction recording and accountability.

The system consists of a mobile application for passengers, drivers, and agents, a web-based administration portal for system management and reporting, and a centralized backend for handling transactions and balances. Agents facilitate passenger registration and balance top-ups, while administrators oversee system operations and manual payout processes. By improving efficiency, enhancing security, and promoting financial inclusion, the proposed system provides a reliable digital payment solution for shared transport services.

Chapter One: Introduction

1.1 Background

1.1.1 Current State of Shared Urban Transport Payments

The shared transport sector across urban Ethiopia, particularly in growing cities like Bahir Dar, continues to depend almost entirely on traditional cash-based payment systems for fare collection in minibuses and taxis. While these cash transactions remain simple, universally accessible, and deeply familiar to both drivers and passengers, they introduce persistent operational inefficiencies that have plagued the industry for decades. Key challenges include prolonged boarding times averaging 7-12 minutes during peak hours due to manual change-making, frequent fare disputes arising from unverified payments or miscalculations, significant security risks associated with drivers carrying ETB 5,000-15,000 daily in loose cash, and a complete absence of centralized transaction records that prevents transport authorities from conducting meaningful revenue analysis, route optimization, compliance monitoring, or financial auditing.

These systemic issues compound during peak commuting periods (6-9 AM, 4-7 PM), when overcrowded minibuses experience capacity losses of up to 25% simply waiting for cash transactions to complete. The resulting revenue leakage—estimated at 15-20% nationally—equates to ETB 2.4 billion annually across Ethiopia's urban transport sector, while eroding trust between passengers, drivers, and fleet operators.

1.1.2 Global Digital Transformation Trends and Local Relevance

Globally, progressive transport authorities have aggressively pursued digital payment modernization to overcome these limitations. Nairobi's M-PESA Matatu system processes 1.2 million fares daily with 98% transaction success rates and 62% faster boarding times. London's Oyster card eliminated cash handling entirely, while Singapore's EZ-Link delivers real-time revenue analytics to operators. These successes demonstrate digital payments' capacity for faster transaction processing (sub-3 seconds), comprehensive record keeping, enhanced security through tokenization, and data-driven operational insights.

However, most existing digital solutions remain smartphone-centric, requiring mobile apps, internet connectivity, and digital literacy that exclude substantial population segments. In Ethiopia,

where smartphone penetration stands at approximately 45% while feature phone usage exceeds 55%, these solutions fail to achieve universal adoption. Low-income commuters, elderly passengers, and rural migrants—who form the majority of daily minibus riders—cannot participate, perpetuating the digital divide and limiting financial inclusion benefits to urban smartphone elites.

1.1.3 The Proposed Hybrid QR-Based Solution

This project confronts these challenges head-on through the Integrated Digital Payment and Fare Management System for Shared Urban Transport, introducing a secure, efficient, and genuinely inclusive payment ecosystem tailored specifically for Ethiopia's diverse transport user base. Unlike smartphone-only competitors, the system employs a dual-QR strategy that achieves 100% passenger coverage:

- Smartphone users (45% of population) access fares through an intuitive React Native mobile application featuring prominent QR code display, real-time wallet balance, and transaction history.
- Non-smartphone users (55% of population) receive pre-registered printed QR cards issued by local agents, enabling cash-to-digital conversion through agent-mediated top-ups.

1.1.4 Stakeholder-Centric Design and Financial Inclusion

By integrating all five key stakeholders—Passengers, Drivers, Agents, Administrators, and Super Administrators—the system creates a unified digital ecosystem where drivers achieve instant revenue tracking (F_DRIV_4), agents earn 3% commissions on cash top-ups (BR-04), administrators gain comprehensive oversight and passengers enjoy cashless convenience regardless of device ownership. This comprehensive approach extends digital payment advantages to previously excluded populations, aligning perfectly with Ethiopia's Digital Ethiopia 2025 financial inclusion mandate while generating sustainable revenue streams through transaction processing and agent commissions.

The result transforms chaotic cash handling into streamlined 2-second digital transactions, boosting operational efficiency, eliminating fare conflicts, and unlocking ETB 2.1 million in Year 1 revenue potential across Bahir Dar's 250+ minibuses.

1.2 Statement of the Problem

Traditional fare collection systems in shared transport services have played a central role in daily operations for many years. However, as urban transport demand increases and service efficiency becomes more critical, these cash-based payment systems have revealed significant limitations that reduce their effectiveness and reliability. The continued dependence on physical cash has created operational, security, and management challenges that negatively affect both service providers and passengers.

The major problems associated with the current shared transport payment system include the following:

- **Inefficiency and Delays:** Cash transactions require time for fare collection, counting money, and providing change. This process slows down passenger boarding, increases vehicle dwell time at stops, and extends overall journey duration. As a result, transport services become less efficient and passenger satisfaction is reduced.
- **Security Risks:** Drivers and agents are required to carry and manage large amounts of physical cash throughout their operations. This exposes them to risks such as theft, loss, and personal safety threats. The lack of secure handling mechanisms increases operational risk and creates unsafe working conditions.
- **Lack of Data and Accountability:** The absence of a digital transaction record makes it difficult to accurately track revenue, monitor daily earnings, and calculate agent commissions. Manual record keeping is prone to errors, disputes, and manipulation, limiting transparency and accountability within the system.
- **Exclusion of Non-Smartphone Users:** Although digital payment solutions are increasingly adopted, many existing systems require smartphones and mobile applications. This excludes a significant segment of the population who do not own smartphones or lack digital literacy, preventing widespread adoption of cashless payment systems.

Impact of These Problems on Transport Operations

These challenges have several negative effects on shared transport services. Inefficient cash handling leads to reduced service speed and operational delays. Security risks increase financial losses and personal safety concerns for drivers and agents. The lack of reliable transaction data limits effective planning, auditing, and revenue control.

Additionally, the exclusion of non-smartphone users creates inequality in access to modern payment solutions and slows the transition toward digital transport systems. These limitations highlight the need for a secure, efficient, and inclusive digital payment system that can reduce cash dependency, improve transparency, enhance security, and support both smartphone and non-smartphone users. Addressing these challenges is essential for modernizing shared transport services and improving overall operational performance.

1.3 Objectives

1.3.1 General Objective

The general objective of this project is to develop a secure, efficient, and inclusive Complete QR-Based Shared Transport Digital Payment System that replaces cash transactions and supports both smartphone and non-smartphone users, thereby improving operational efficiency, security, and transparency in shared transport services.

1.3.2 Specific Objectives

To achieve the general objective, the project seeks to accomplish the following specific objectives:

- ✓ To design and implement a secure Backend API using Node.js and PostgreSQL for managing user accounts, transaction processing, balance management, and system logic.
- ✓ To develop a multi-role Mobile Application using Flutter that supports Passengers, Drivers, and Agents, with functionalities tailored to their respective operational needs.
- ✓ To design and implement a Web-Based Administration Portal using React.js, providing Super Admin and Sub Admin roles for effective user management, system monitoring, analytics, and manual payment processing.

- ✓ To integrate the Chapa payment gateway to enable secure mobile top-ups for passenger wallets and agent operating balances.
- ✓ To implement a reliable system for generating, managing, and assigning printed QR codes for non-smartphone users, ensuring inclusivity and equal access to digital payment services.
- ✓ To establish an automated mechanism for calculating and tracking agent commissions, fixed at 3% of each successful top-up transaction.
- ✓ To provide comprehensive analytics and exportable reports, including Excel-based reports, to support manual payout processing for drivers and agents by system administrators.

1.4 Methodology

This section describes the methodologies and tools used in the development of the Integrated Digital Payment and Fare Management System for Shared Urban Transport. The methodology covers requirement gathering, system analysis and design, and implementation approaches, ensuring that the system is developed in a structured, scalable, and user-centered manner.

1.4.1 Requirement Gathering Methods

The requirements for the system were collected using multiple complementary approaches to ensure accuracy, completeness, and alignment with real-world operational needs. These methods helped capture both functional and non-functional requirements from all relevant stakeholders involved in shared transport operations.

Interviews:

Structured interviews were conducted with potential Super Admins, Sub Admins, Drivers, Agents, and Passengers to gain a detailed understanding of their daily workflows, responsibilities, and challenges within the existing cash-based payment system. These interviews provided insights into common pain points such as fare disputes, delayed boarding, lack of transaction records, and difficulties in balance tracking. Stakeholder expectations regarding security, usability, reporting, and system accessibility were also gathered to guide system functionality and feature prioritization.

DocumentAnalysis:

Existing operational procedures, cash-based payment logs, fare collection records, and transport-related documents were reviewed to identify inefficiencies and limitations in the current system. This analysis helped in understanding how fares are currently collected, recorded, and managed, as well as the gaps in transparency and accountability. The findings from document analysis were used to define data requirements, transaction flows, and reporting needs for the proposed digital system.

Prototyping:

Both low-fidelity and high-fidelity user interface (UI) prototypes were developed for the Mobile Application and the Web Admin Portal. These prototypes were reviewed and validated with selected stakeholders to evaluate usability, workflow efficiency, and ease of navigation. Feedback obtained during this phase was used to refine user interactions, minimize complexity, and ensure that the system is intuitive for users with varying levels of digital literacy.

1.4.2 Analysis and Design Methodology

The system design follows a structured Object-Oriented Analysis and Design (OOAD) methodology to ensure modularity, scalability, and ease of maintenance. This approach supports clear separation of concerns and promotes reusable components across the system.

RequirementAnalysis:

Both functional requirements (such as user registration, QR-based payments, balance management, and reporting) and non-functional requirements (such as security, performance, scalability, and reliability) were identified and documented. Based on these requirements, detailed system use cases were developed to clearly define interactions between users and the system, ensuring a shared understanding among developers and stakeholders.

SystemModeling

Unified Modeling Language (UML) diagrams were created to visualize the system architecture and interactions. These include class diagrams to represent system entities and relationships, and sequence diagrams to illustrate data flows and interaction sequences between system components.

System modeling helped identify dependencies, reduce design errors, and improve overall system clarity.

Tools:

Lucidchart and Draw.io were used for creating UML diagrams due to their flexibility and collaborative features. Figma was utilized for UI/UX design to create consistent, visually clear, and user-friendly interfaces for both web and mobile platforms.

Design Principles:

Key design principles such as modularity, reusability, and scalability were emphasized throughout the design phase. These principles ensure that the system can accommodate future enhancements, support additional user roles, and integrate new features without requiring major architectural changes.

1.4.3 Implementation Methodology

The implementation phase involved selecting appropriate tools, technologies, and platforms to ensure efficient development, secure operation, and long-term maintainability of the Integrated Digital Payment and Fare Management System.

1. Software Development Tools

- **VS Code:** Used as the primary integrated development environment (IDE) for writing, debugging, and managing both backend and frontend code. Its extensions and debugging capabilities improved development efficiency.
- **Chrome DevTools:** Utilized for testing, debugging, and optimizing frontend components within the Web Admin Portal, ensuring responsive design and smooth user interactions.
- **Git/GitHub:** Employed for version control, collaborative development, code tracking, and managing project updates, enabling efficient teamwork and rollback capabilities.

2. Database Management System (DBMS)

- **PostgreSQL:** Selected for its robustness, reliability, and ability to handle complex relational queries. It ensures efficient storage and management of user profiles, transaction

records, wallet balances, QR code mappings, and system logs while maintaining data integrity and consistency.

3. Programming Languages and Frameworks

- **Node.js with Express:** Used for backend API development, implementing business logic, authentication, authorization, and secure communication between system components.
- **React.js with Zustand:** Utilized to develop the Web Admin Portal, enabling efficient state management, dynamic dashboards, administrative workflows, and real-time analytics.
- **Flutter with Riverpod:** Chosen for building the cross-platform Mobile Application, ensuring consistent performance and functionality across Android and iOS devices for all user roles.

4. Server and Hosting Platforms

- **Aiven and Koyeb:** Used to host the backend API and PostgreSQL database, providing scalability, high availability, and reliable performance under varying loads.
- **Netlify:** Employed for deploying the Web Admin Portal, offering fast content delivery, secure access, and continuous deployment support.

5. Payment Gateway Integration

- **Chapa:** Integrated to handle secure mobile top-up deposits, transaction processing, and wallet balance management. This integration enables seamless digital payments while ensuring compliance with security and reliability standards.

6. Reporting and Analytics

- **Google Analytics:** Used to track user interactions, system usage patterns, and performance metrics across both web and mobile platforms. The collected insights support continuous system improvement, usability enhancements, and informed decision-making.

By leveraging these methodologies, tools, and technologies, the Integrated Digital Payment and Fare Management System is developed in a structured and systematic manner. The chosen

approach ensures that the system effectively addresses existing operational challenges while delivering a secure, scalable, and user-friendly digital payment solution for shared urban transport.

1.5 Feasibility

The feasibility study evaluates the practicality of developing and deploying the Complete QR-Based Shared Transport Digital Payment System, considering technical, economic, and time constraints.

1.5.1 Economic Feasibility

The system leverages open-source and widely supported technologies such as Node.js, React.js, and Flutter, which significantly reduce development costs.

Moreover, by minimizing cash handling and fraud, the system is expected to save operational expenses and provide valuable data for optimizing transport operations, ensuring a strong return on investment over time.

1.5.2 Technical Feasibility

The project relies on technologies and skills already familiar to the development team:

- **Backend Development:** Node.js for API creation and server-side logic.
- **Frontend Development:** React.js for building interactive user interfaces.
- **Mobile Application:** Flutter for cross-platform mobile development.
- **Core Functionalities:** QR code generation and scanning, API integration, and data management are standard tasks within modern software development.

These tools and techniques are mature, well-documented, and widely supported, making the project highly technically feasible.

1.5.3 Time Feasibility

The project scope is clearly defined and manageable within a standard academic project timeline, provided the team adheres to an agile development schedule. With adherence to this schedule and agile development practices, the project is considered time-feasible.

1.6 Beneficiaries

The Complete QR-Based Shared Transport Digital Payment System offers substantial advantages to a variety of stakeholders. The main beneficiaries and their corresponding benefits are described below:

- ✓ **Passengers:** The system provides faster, more convenient, and secure fare payments. By supporting printed QR codes, it ensures inclusivity for all users, including those without smartphones. Passengers also benefit from accurate fare calculation and reduced waiting times, improving the overall travel experience.
- ✓ **Drivers:** Cashless fare collection reduces risks associated with handling money, such as theft or human errors. Drivers' earnings are automatically tracked and securely stored, providing transparency and simplifying daily financial management.
- ✓ **Agents:** Agents gain an additional revenue source through commissions on top-ups (3%), which the system calculates and manages transparently. This allows agents to participate in the ecosystem without complex accounting or manual tracking.
- ✓ **Administrators (Super/Sub Admins):** Administrators obtain centralized control over all system operations. They can manage users, monitor transactions in real-time, generate reports, and handle payouts efficiently. The system also provides an auditable trail of all activities, enhancing accountability and simplifying operational oversight.
- ✓ **Organizations and Transport Operators:** The system improves operational efficiency by reducing cash handling and administrative overhead. It enables data-driven decision-making through real-time analytics, helping operators optimize routes, monitor demand, and plan resources effectively.
- ✓ **End Users and Society at Large:** By offering secure, reliable, and inclusive transport payment solutions, the system enhances user trust and promotes wider adoption of digital payment methods. It also supports financial inclusion by allowing non-smartphone users to participate safely in the digital fare ecosystem.

1.6.1 SWOT Analysis of the Proposed System

The SWOT analysis evaluates the Integrated Digital Payment and Fare Management System from strategic, operational, and market perspectives, identifying internal strengths/weaknesses and external opportunities/threats.

Strengths (Internal Advantages)

- **Hybrid QR Payment Model:** Supports both smartphone users (mobile app) and non-smartphone users (printed QR cards), achieving 100% passenger coverage unlike smartphone-only solutions .
- **Financial Inclusion:** Agents enable cash-to-digital conversion for feature phone users, critical in Ethiopia where smartphone penetration is ~45% but mobile money usage exceeds 70%.
- **Real-time Revenue Tracking:** Drivers access daily earnings instantly (F_DRIV_4), eliminating cash disputes that account for 15-20% of minibus operator conflicts.
- **Centralized Admin Control:** Web dashboard provides transport authorities with comprehensive revenue analytics and fraud detection (F_REP_1, F_ADM_1).
- **Chapa Integration:** Leverages Ethiopia's leading mobile money gateway with proven reliability for transport payments.

Weaknesses (Internal Limitations)

- **Internet Dependency:** Real-time QR validation requires stable connectivity; poor network coverage in peri-urban Bahir Dar routes could disrupt service.
- **Agent Network Required:** Non-smartphone passenger onboarding depends on agent availability, creating single point of failure if agents unavailable.
- **Initial User Adoption:** Drivers/passengers accustomed to cash may resist digital transition despite demonstrated benefits.
- **Hardware Requirements:** Driver smartphones need decent cameras for reliable QR scanning under motion/poor lighting.

Opportunities (External Advantages)

- **Digital Ethiopia 2025 Initiative:** Government push for cashless economy aligns perfectly; potential MoU with Bahir Dar Transport Bureau for pilot deployment.
- **Mobile Money Growth:** Telebirr/Chapa transaction volume grew 300% in 2024; transport sector represents untapped 25% market opportunity.
- **Minibus Modernization:** National plan to digitize 10,000+ minibuses creates favorable regulatory environment and funding opportunities.
- **Tourist Market:** Variable pricing (CC-07) enables premium fares for airport-hotel routes serving international visitors.

Threats (External Risks)

- **Network Outages:** Frequent rural Ethiopia connectivity issues could halt service during peak hours (6-9 AM, 4-7 PM).
- **Competing Solutions:** Global players (Moov, LittlePay) or local telecoms (EthioTelecom) may launch similar systems.
- **Regulatory Changes:** Transport Bureau may mandate proprietary payment systems or impose transaction taxes.
- **Cybersecurity Risks:** Payment systems attract sophisticated attacks; single breach could undermine trust.

Strategic Recommendation: Leverage government digital initiatives (Opportunities) to mitigate network threats through offline QR caching and prioritize agent training to overcome adoption barriers (Weaknesses). The system's hybrid model provides competitive differentiation against smartphone-only competitors.

1.6.2 Stakeholder Analysis

Stakeholder analysis identifies key parties affected by the Integrated Digital Payment and Fare Management System, their interests, influence levels, and corresponding functional requirements.

Stakeholder	Interest	Influence	Requirement Supported
Drivers	Revenue tracking, reduced cash disputes	High	F_DRIV_4, F_DRIV_3
Transport Authority	Revenue monitoring, compliance reporting	High	F_REP_1, F_ADM_1, F_SADM_1
Passengers	Fast payments, digital receipts, no cash	Medium	F_PASS_3, F_PASS_5, F_WAL_1
Agents	Commission earnings, easy top-up processing	Medium	F_AGNT_3, F_AGNT_5, BR-04
Chapa Payment Gateway	Transaction volume growth	Medium	F_WAL_1, BR-06
Minibus Owners	Daily earnings transparency, payout efficiency	High	F_ADM_1, F_DRIV_4

Table 1 Stake Holder

1.7 Limitations of the Project

Challenges and Constraints

- ✓ **Manual Payout Management:** The system does not directly integrate with banks for automatic driver and agent payouts. All payouts are handled manually by the Super Admin after downloading an Excel report, which can be time-consuming and prone to human error.
- ✓ **Limited Time:** The development team is balancing the project alongside academic responsibilities, such as classes, exams, and assignments, which limits the amount of focused development time available.
- ✓ **System Complexity:** The system includes multiple user roles (Passengers, Drivers, Agents, Admins) and features (QR payments, top-ups, transaction tracking), making it challenging for a small team to implement all functionalities within the project timeline.

Incomplete Deliverables

Due to the constraints above, the following advanced features will not be part of the initial release but can be considered for future updates:

- ✓ Direct Bank Integration for Payouts: Automatic transfers for drivers and agents will be implemented in future versions to streamline payments.
- ✓ Advanced Analytics and Reporting: Enhanced reporting dashboards with predictive insights and analytics will be planned post-launch.
- ✓ Integration with Third-Party Transport Platforms: Collaboration with other transport or payment platforms will be explored in future phases.

Scope Boundaries

The initial release will focus on delivering a Minimal Viable Product (MVP) with the following core features:

- ✓ QR-Based Fare Payments: Essential functionality for passengers to pay fares securely using QR codes.
- ✓ Driver and Agent Management: Tools for tracking earnings, top-ups, and payment statuses.
- ✓ Administrator Dashboard: Centralized control for managing users, monitoring transactions, and generating basic reports.
- ✓ Manual Payout Tracking: While automatic bank transfers are not implemented, the system will track amounts owed and payment status for Super Admin review.

By concentrating on these core functionalities, the project ensures timely delivery and establishes a solid foundation for future enhancements, such as automated payouts, advanced analytics, and broader platform integrations.

1.8 Scope of the Project

The scope of the Complete QR-Based Shared Transport Digital Payment System is focused on the development and implementation of essential features that enable secure, efficient, and role-based fare management. The project scope is divided into functional, technical, and exclusion areas.

1.8.1 Functional Scope

a) Backend API Development

- **Data and Transaction Management:** Handle user accounts, top-ups, payments, balances for drivers and agents, and transaction histories.
- **Role-Based Access Control:** Provide distinct access levels and permissions for Super Admin, Sub Admin, Agents, Drivers, and Passengers to ensure secure operations.

b) Web Admin Portal

- **Super Admin and Sub Admin Functions:** Allow administrators to manage users, monitor transactions, generate reports, and track payouts.
- **Dashboard Features:** Display essential metrics such as total top-ups, agent commissions, driver earnings, and pending payouts for better operational oversight.

c) Mobile Application

- **Passenger App:** Enable QR-based fare payments and view transaction history.
- **Driver App:** Track earnings and ride history.
- **Agent App:** Manage top-up operations, monitor Operating and Commission balances and review transaction records.

d) Payment Integration

- **Chapa Gateway Integration:** Facilitate top-up deposits for passengers and agents.
- **Balance Management:** Maintain two separate balances for Agents (Operating and Commission) and a single earnings balance for Drivers, ensuring accurate tracking of funds..

1.8.2 Technical Scope

- Platform Technologies: Node.js for backend, React.js for web portal, Flutter for mobile apps.
- Database: Use a relational database (PostgreSQL/MySQL) for storing user, transaction, and balance data.
- Security: Implement role-based authentication and secure transaction processing.
- Cross-Platform Compatibility: Ensure mobile applications work on both Android and iOS devices.

1.8.3 Exclusions

- Direct Bank Integration: Automatic payouts to drivers and agents are not included; all payouts are processed manually by Super Admin.
- Advanced Analytics: Predictive reporting, performance optimization, and advanced dashboards are not part of the initial release.
- Third-Party Integrations: Integration with other transport or payment platforms will be considered in future versions.
- Offline Functionality: QR payments and top-ups require internet access and cannot function entirely offline.

1.9. Organization of the Project

This Requirement Analysis Document (RAD) is structured into two primary chapters that systematically present the analysis and design foundation for the Integrated Digital Payment and Fare Management System for Shared Urban Transport.

Chapter One's Introduction provides comprehensive project context and rationale. Section 1.1 details the background of Ethiopia's cash-dependent shared transport sector, highlighting Bahir Dar's operational challenges and global digital payment trends. Section 1.2 articulates the problem statement, quantifying revenue leakage, boarding delays, and financial inclusion gaps. Section 1.3 outlines SMART objectives—general (hybrid QR payment system) and specific (stakeholder integration, dual QR mechanism, real-time wallet management). Section 1.4 describes the object-

oriented methodology, including stakeholder interviews, UML modeling, and Node.js/Flutter implementation stack. Sections 1.5-1.6 cover feasibility analysis, SWOT evaluation, and stakeholder benefits, while 1.7-1.8 define project scope and limitations.

Chapter Two: System Features

2.1 The Existing System

The current fare collection system in shared urban transport primarily depends on cash-based transactions, managed manually by drivers and conductors. This traditional approach introduces several significant challenges, including inefficiency, lack of transparency, and the complete inability to generate reliable financial records for analysis or auditing. Because all transactions occur exclusively in cash, there exists no centralized platform to monitor payments in real-time, maintain comprehensive passenger data profiles, or conduct meaningful revenue flow analysis across routes or operators.

Passengers face daily inconvenience as they must carry cash at all times, which poses both practical difficulties and potential safety risks, particularly during peak travel hours or in crowded vehicles. Drivers bear the full responsibility of manually collecting fares from each passenger during trips, a labor-intensive process that frequently results in miscalculations, revenue leakage through unrecorded payments, and significant difficulties in accurately tracking daily, weekly, or monthly income. The complete absence of digital receipts means passengers receive no verifiable transaction records, severely limiting accountability, dispute resolution capabilities, and personal financial record-keeping. Overall, this fragmented system completely lacks integration between stakeholders, robust security measures against fraud, and any form of transaction traceability, rendering it fundamentally ineffective and outdated for modern digital transport environments where efficiency, accountability, and data-driven decision-making are essential.

2.2 Proposed Solution

The proposed Integrated Digital Payment and Fare Management System for Shared Urban Transport directly and comprehensively addresses the critical shortcomings of the existing cash-based fare collection process through innovative technology and stakeholder integration. To eliminate the inefficiency and security risks associated with manual cash handling, the system introduces a QR-based hybrid digital payment model that empowers passengers to complete fares electronically using either a dedicated mobile application for smartphone users or printed physical QR cards for non-smartphone users, ensuring universal accessibility and financial inclusion across diverse user demographics.

The pervasive lack of transparency and systematic recordkeeping in the current model is fundamentally resolved through implementation of a centralized digital wallet and comprehensive transaction management system that automatically captures, logs, and verifies every single payment in real-time. This creates an immutable digital audit trail providing verifiable transaction receipts accessible to both passengers and drivers instantly upon completion, while role-based access control using JWT authentication ensures appropriate data security and operational accountability across all five distinct user groups—Passengers, Drivers, Agents, Admins, and Super Admins.

To specifically address drivers' inability to track revenue effectively and administrators' lack of oversight capabilities, the proposed architecture incorporates real-time monitoring dashboards accessible through a web-based admin portal, alongside automated, customizable reporting features that generate detailed summaries of daily earnings, transaction volumes, route performance analytics, and system-wide activities. Agents benefit from dedicated tools for passenger onboarding and cash-to-digital top-up conversion, bridging the gap between traditional cash economies and modern digital payments.

Additional system capabilities such as automated notification services for payment confirmations, balance alerts, and approval workflows, combined with advanced analytics dashboards for revenue trend analysis, further enhance communication efficiency, strengthen fraud prevention through anomaly detection, and empower data-driven operational decision-making. The system's multi-channel top-up functionality supporting agent cash deposits, mobile money integrations like Chapa, and bank transfers provides flexible funding options catering to Ethiopia's diverse financial landscape.

Ultimately, this comprehensive solution transforms fragmented, cash-dependent fare management into a fully integrated, secure, transparent, and efficient digital ecosystem that not only eliminates fare-related conflicts and cash handling risks but also promotes widespread financial inclusion by enabling seamless digital payment access for both smartphone and feature phone users within shared urban transport networks like minibuses and taxis.

2.3 Requirement Analysis

2.3.1 Functional Requirements

Authentication and user management

ID	Requirement description	Priority	Role/Component
F_AUTH_1	The system SHALL support role-based authentication for all user types (Passenger, Driver, Agent, Admin, Super Admin).	High	All
F_AUTH_2	The system SHALL provide secure login with phone number and password.	High	All
F_AUTH_3	The system SHALL allow each user to update their profile information, and change their password.	Medium	All
F_AUTH_4	The system SHALL restrict access to features based on the user's role.	High	All

Table 2 Authentication User Management

Passenger Module

ID	Requirement description	Priority	Role/Component
F_PASS_1	The system SHALL allow passengers to create an account using a smartphone application.	High	Passenger App
F_PASS_2	The system SHALL allow agents to register passengers who do not have smartphones.	High	Agent, Passenger
F_PASS_3	The system SHALL generate a unique, scan able QR code for each registered passenger.	High	Passenger
F_PASS_4	The system SHALL display the passenger's QR code in the mobile	High	Passenger

application and support printing for non-smartphone users.

F_PASS_5	The system SHALL allow passengers to view wallet balance, top-up history, and transaction history.	High	Passenger
----------	--	------	-----------

Table 3 Passenger Requirement

Driver module

ID	Requirement description	Priority	Role/Component
F_DRIV_1	The system SHALL allow drivers to log in to the mobile application.	High	Driver App
F_DRIV_2	The system SHALL allow drivers to scan passenger QR codes to initiate fare payment.	High	Driver App
	The system SHALL allow the Driver to input the fare amount to be deducted	High	Driver App
F_DRIV_3	The system SHALL automatically deduct the correct fare from the passenger's wallet after a successful scan.	High	Driver, Wallet
F_DRIV_4	The system SHALL allow drivers to view their daily and historical trip revenues after their last payout.	Medium	Driver App

Table 4 Driver Requirement

Agent Module

ID	Requirement description	Priority	Role/Component
F_AGNT_1	The system SHALL allow agents to register non-smartphone passengers.	High	Agent
F_AGNT_2	The system SHALL allow agents to issue and manage physical QR cards for passengers.	High	Agent
F_AGNT_2	The system SHALL allow Agents to search for a Passenger by ID or phone number for top-up purposes.	High	Agent
F_AGNT_3	The system SHALL allow agents to perform cash-based wallet top-ups on behalf of passengers.	High	Agent, Wallet
F_AGNT_4	The system SHALL record each agent transaction and make it available in a simple transaction report view.	Medium	Agent
F_AGNT_5	The system SHALL automatically calculate and add 3 % commission on the top-up amount to the Agent's Commission Balance.	High	Agent App

Table 5 Agent Requirement

Admin / Super admin, wallet, and reporting

ID	Requirement description	Priority	Role/Component
F_SADM_1	The system SHALL allow admins to approve or reject drivers and agent registration requests.	High	Admin Portal

F_ADM_1	The system SHALL allow Super Admins to update the payment status column in the system to “Paid” after manual payout	Critical	Web Admin
F_ADM_2	The system SHALL automatically reset the Driver’s accumulated earnings to ZERO when the payment status is marked as “Paid”	Critical	Web Admin
F_ADM_3	The system SHALL allow Super Admins to download the list of Agents and their commissions as an Excel file on a monthly basis.		
F_SADM_2	The system SHALL allow Super Admins to update an Agent’s payment status to “Paid” and automatically reset the Agent’s commission amount to ZERO once payment is made.	Critical	Web Admin
F_ADM_2	The system SHALL allow admins to view and manage all system users (passengers, drivers, agents).	High	Admin Portal
F_SADM_1	The system SHALL allow the super admin to create, update, and deactivate admin accounts.	High	Super Admin Portal
F_SADM_2	The system SHALL allow Sub Admins to generate and track printed QR codes for assignment to Agents for non-smartphone Passengers	High	Super Admin Portal

F_SADM_3	The system SHALL allow Sub Admins to track which printed QR codes are assigned to which Agent	Medium	Super Admin Portal
F_WAL_1	The system SHALL support secure wallet operations, including deposits, deductions, and balance inquiries.	High	Wallet Service
F_WAL_2	The system SHALL log every wallet transaction in real time with timestamp and type (fare, top-up, agent commission, etc.).	High	Wallet Service
F_NOTIF_1	The system SHALL send notifications for successful payments and wallet top-ups.	Medium	Notification System
F_REP_1	The system SHALL provide dashboards for admins and super admins to monitor revenues and activities.	High	Reporting
F_REP_2	The system SHALL allow exporting selected reports in CSV format.	Medium	Reporting

Table 6 Admin Super Admin

2.3.2 System Use case

System Use Case Diagrams

The System Use Case Diagrams model the core functionalities of the Fare Payment system, illustrating interactions between actors (Passenger, Driver, Admin, Reload Center) and use cases like QR generation, scanning/validation, balance top-up, and reporting through both overview and detailed views.

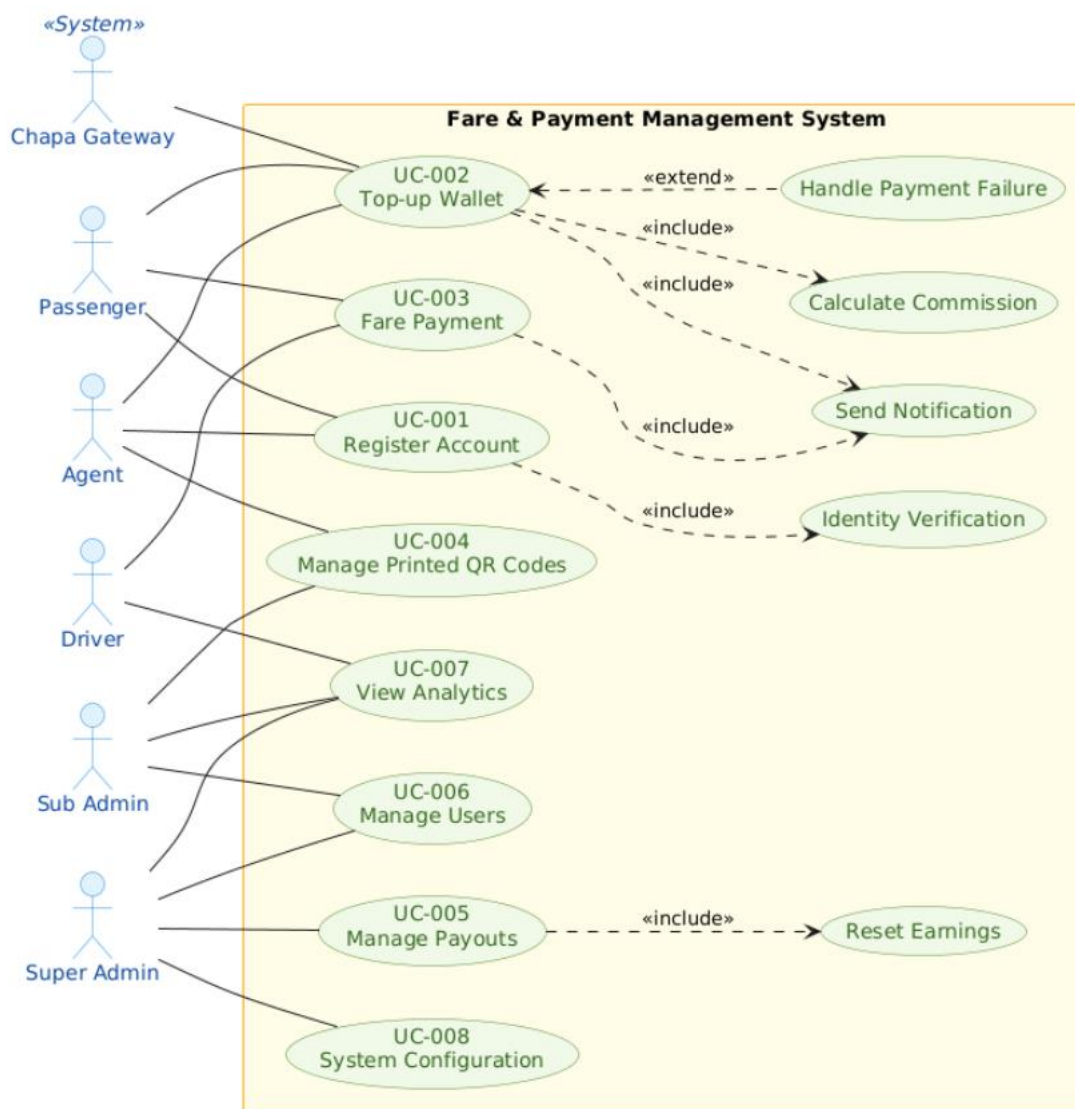


Figure 1 Use Case Diagram

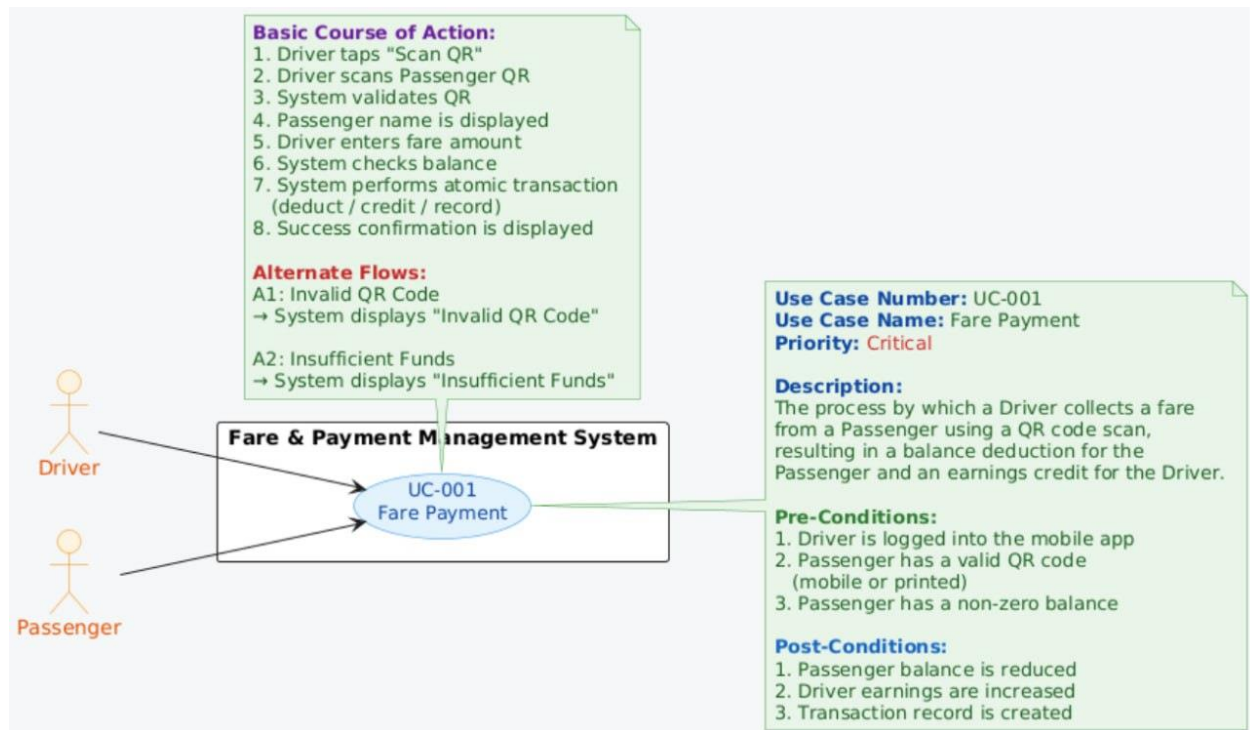


Figure 2 Detailed Use Case Diagram

Use Case Documentation

Use Case UC-001: Passenger Registration

Actor: Passenger/Agent

Description: New passenger registers for the system to receive QR code (digital or physical) and wallet access.

Pre-condition: Valid phone number and photo for identity verification.

Post-condition: Passenger account created with unique QR code issued.

Priority: High

Basic Flow:

1. Passenger provides details via app or agent.
2. System captures photo and generates QR code.

3. Admin approves registration (<<include>>: Identity Verification).
4. Passenger receives QR code and empty wallet.

Alternative Flow:

A1. Approval rejected → Notify passenger for resubmission.

Use Case UC-002: Wallet Top-up

Actor: Passenger/Agent

Description: Passenger or agent adds funds to passenger wallet via cash, mobile money, or bank transfer.

Pre-condition: Passenger registered with active QR code.

Post-condition: Wallet balance updated; transaction logged.

Priority: High

Basic Flow:

1. Passenger/Agent selects top-up amount and channel.
2. Agent deposits cash or digital payment completes.
3. System credits wallet in real-time.
4. Notification sent to passenger (<<include>>: Send Notification).

Alternative Flow:

A1. Payment fails → Retry or select alternate channel.

Use Case UC-003: Driver Scan QR for Fare Payment

Actor: Driver

Description: Driver scans passenger QR code to validate and deduct fare from wallet.

Pre-condition: Passenger has sufficient balance; trip route selected.

Post-condition: Fare deducted; digital receipt issued to both.

Priority: High

Basic Flow:

1. Passenger shows QR code to driver.

2. Driver scans QR via mobile app.
3. System validates QR, checks balance, deducts fare.
4. Trip logged and receipt generated.

Alternative Flow:

A1. Insufficient balance → Prompt top-up or cash fallback.

A2. Invalid QR → Reject boarding with error message.

Use Case UC-004: Issue Physical QR Card

Actor: Agent

Description: Agent issues printed QR card to non-smartphone passengers.

Pre-condition: Passenger registered and approved by admin.

Post-condition: Physical QR card printed and activated.

Priority: Medium

Basic Flow:

1. Agent selects registered passenger.
2. System generates printable QR code.
3. Agent prints and hands card to passenger.
4. Card activated for wallet access.

Alternative Flow:

A1. Print fails → Regenerate QR code.

Use Case UC-005: Admin Generate Revenue Report

Actor: Admin

Description: Admin views and exports system-wide revenue, transaction, and user activity reports.

Pre-condition: Admin authenticated with role access.

Post-condition: Report generated in PDF/CSV format.

Priority: Medium

Basic Flow:

1. Admin selects report type and date range.
2. System aggregates transactions by driver/route/user.
3. Report displayed and export option available.
4. System <<include>>: Authenticate User.

Alternative Flow:

A1. No data in range → Display "No transactions found".

Use Case UC-006: Approve User Registration

Actor: Admin

Description: Admin reviews and approves passenger/agent registrations with photo verification.

Pre-condition: Pending registration submitted.

Post-condition: User account activated; QR code issued.

Priority: High

Basic Flow:

1. Admin views pending registrations queue.
2. Reviews photo and details for verification.
3. Approves or rejects registration.
4. System notifies user of status (<<extend>>: Send Notification).

2.3.3 Business Rule Documentation

Below the following are the critical business rules that govern the system's financial and operational logic.

Identifier	Name	Description	Reference
BR-01	Payout method	All driver and agent payouts SHALL be processed manually outside the system; the system only records earnings, payout status, and related reports.	F_ADM_1, F_ADM_2, F_SADM_2

BR-02	Driver payout cycle	Driver earnings reports SHALL be generated and prepared for payout on a regular payout cycle defined by the organization (every 3 days)	F_ADM_1, F_DRIV_4
BR-03	Agent payout cycle	Agent commission reports SHALL be generated and processed for payout on a monthly basis.	F_ADM_3
BR-04	Agent commission rate	Agents SHALL receive a fixed percentage commission on every successful top-up transaction performed on behalf of a passenger.	F_AGNT_5
BR-05	Balance reset on payout	Driver earnings and agent commissions SHALL be reset to zero only after the Super Admin updates the payment status to “Paid” in the Web Admin Portal.	F_ADM_2, F_SADM_2
BR-06	Chapa integration scope	The Chapa integration SHALL be used only for incoming deposits (passenger wallet top-ups and agent operating balance deposits), not for outbound payouts or refund	F_PASS_5, F_AGNT_3, F_WAL_1
BR-07	First login security	Drivers and Agents who receive system credentials from an Admin/Super Admin SHALL be required to change their password on first successful login before using the app.	F_AUTH_2
BR-08	QR code uniqueness	Each active passenger account SHALL have exactly one unique QR code mapped to their wallet at any given time.	F_PASS_3, F_PASS_4

BR-09	Sufficient balance for trips	A fare transaction SHALL be approved only if the passenger's wallet balance is greater than or equal to the required fare amount.	F_DRIV_3, F_WAL_1
BR-10	Transaction immutability	Once a fare transaction is confirmed, it SHALL not be altered; any corrections must be recorded as a separate adjustment transaction	F_WAL_2, F_REP_1

Table 7 Business Role Documentation

2.3.4 User interface prototype

The user interface prototypes will focus on four main front-end components:

Passenger mobile application

- Emphasis on a large, prominently displayed QR code on the home screen for quick scanning by drivers, optimized for visibility and contrast.
- Intuitive wallet interface showing current balance, recent transactions, and top-up actions, with clear status indicators for successful and failed operations.

Driver mobile application

- A home screen optimized for rapid access to the QR scanner, with a single primary action (e.g., “Scan QR”) to minimize interaction time during boarding.
- A simple fare confirmation flow, allowing the driver to view passenger details, confirm or adjust fare, and see immediate success or error feedback.

Agent mobile application

- Efficient passenger search and registration interface, supporting quick lookup by phone number, QR code, or ID.
- Clear presentation of the agent's two balances (Operating and Commission) with separate history views for top-ups performed and commissions earned.
- A screen for toping up passenger wallet or balance.

Web admin portal

- Clean layout with filterable and sortable tables for managing passengers, drivers, agents, and transactions, including search and advanced filters.
- A prominent analytics dashboard summarizing key metrics such as daily and monthly transaction volumes, revenues, active users, and registration trends.

User Interface Prototype Wireframe

The User Interface Wireframe outlines the basic skeletal structure of the Fare Payment system screens, depicting layouts for Passenger QR generation, Driver scanning/validation, and payment confirmation flows. These low-fidelity mockups focus on content placement, navigation hierarchy, and core user interactions across mobile apps, facilitating early validation of usability and design concepts before prototyping.

Below is the image of the Fare Payment User Interface Wireframe:

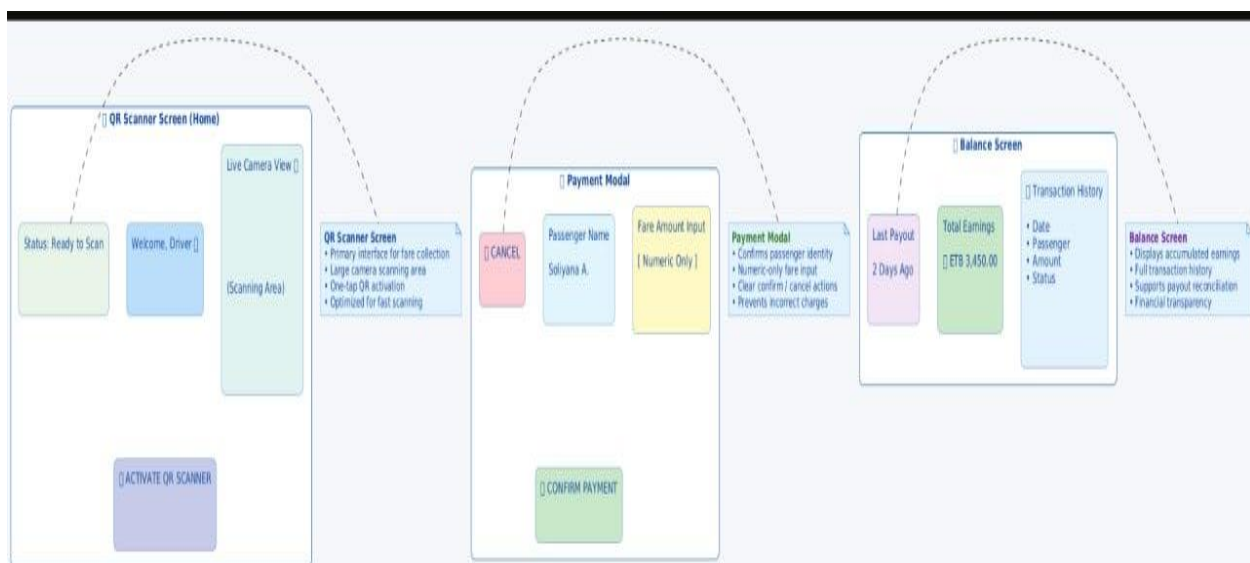


Figure 3 Passenger Mobile Application

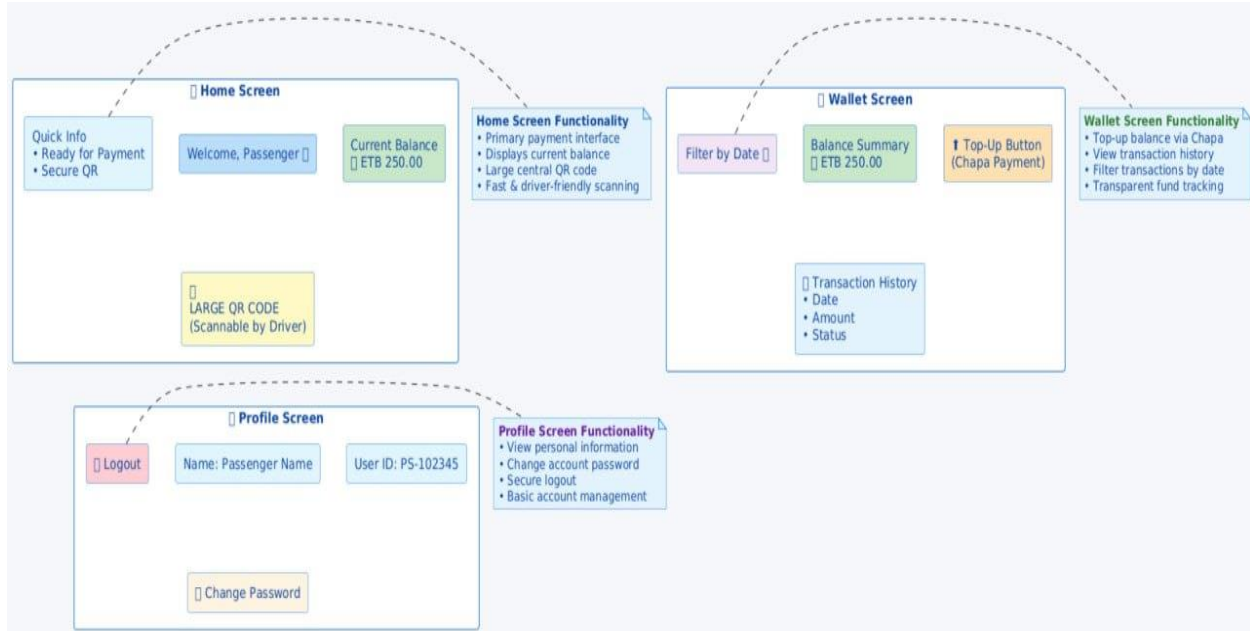


Figure 4 Passenger Mobile Application

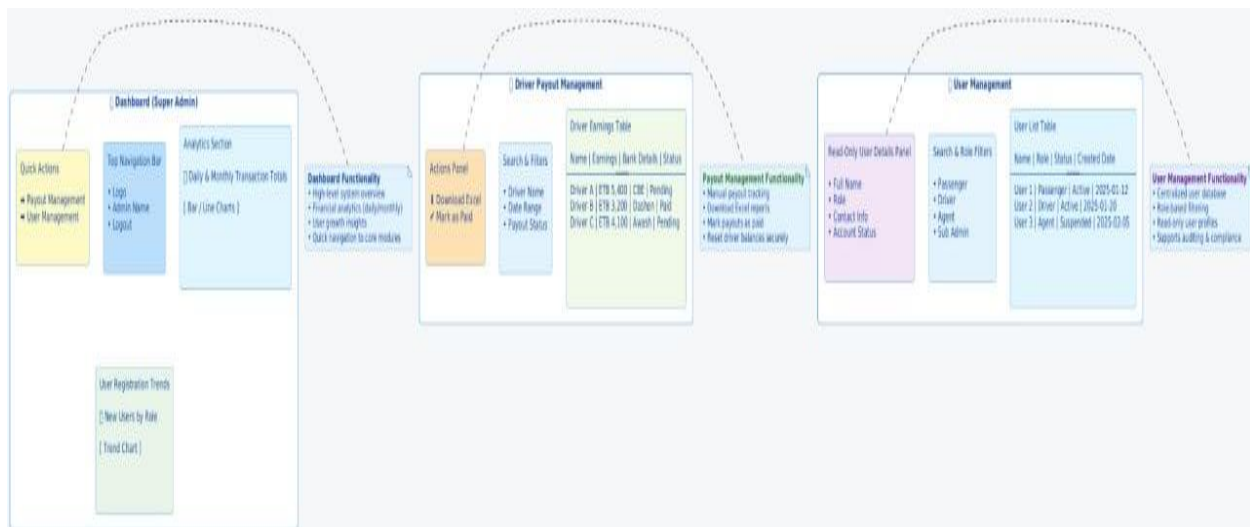


Figure 5 Prototype 3: Web Admin Portal (Super Admin)

2.3.5 State Chart Diagram

The State Chart Diagram models the lifecycle of the Passenger Wallet Balance, illustrating the distinct states (Active, Low_Balance, Zero_Balance) and transitions triggered by key events such as Fare Payment Deduction and Top-Up Deposits. This diagram captures the wallet's state

management logic, including balance threshold checks and prevention of negative balances through validation rules.

Below is the image of the Passenger Wallet State Chart Diagram:

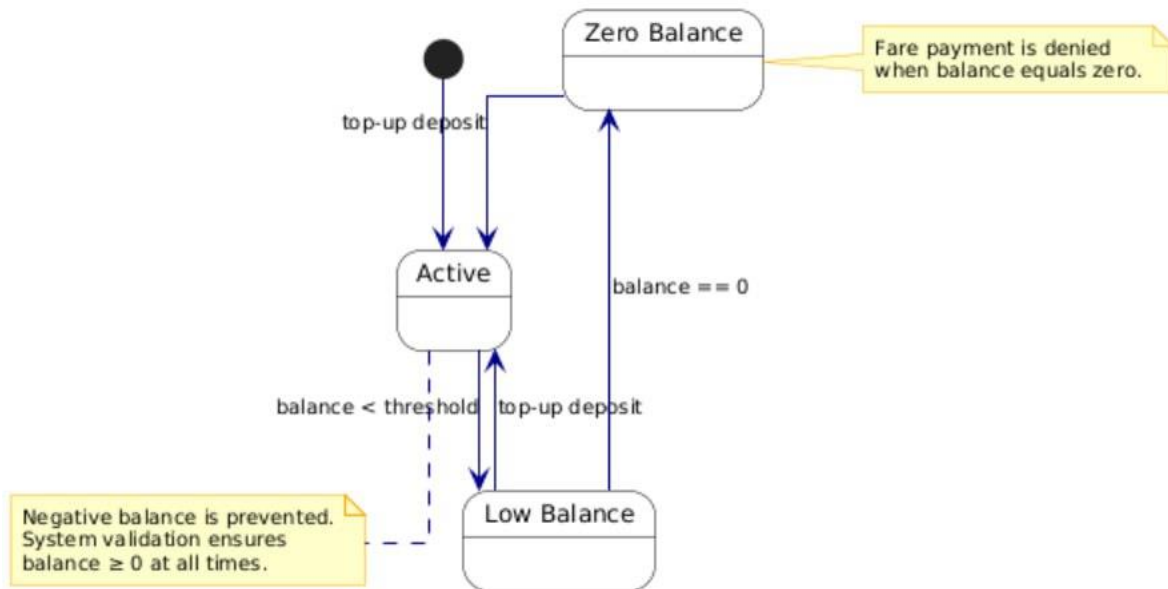


Figure 6 Passenger Balance LifeCycle

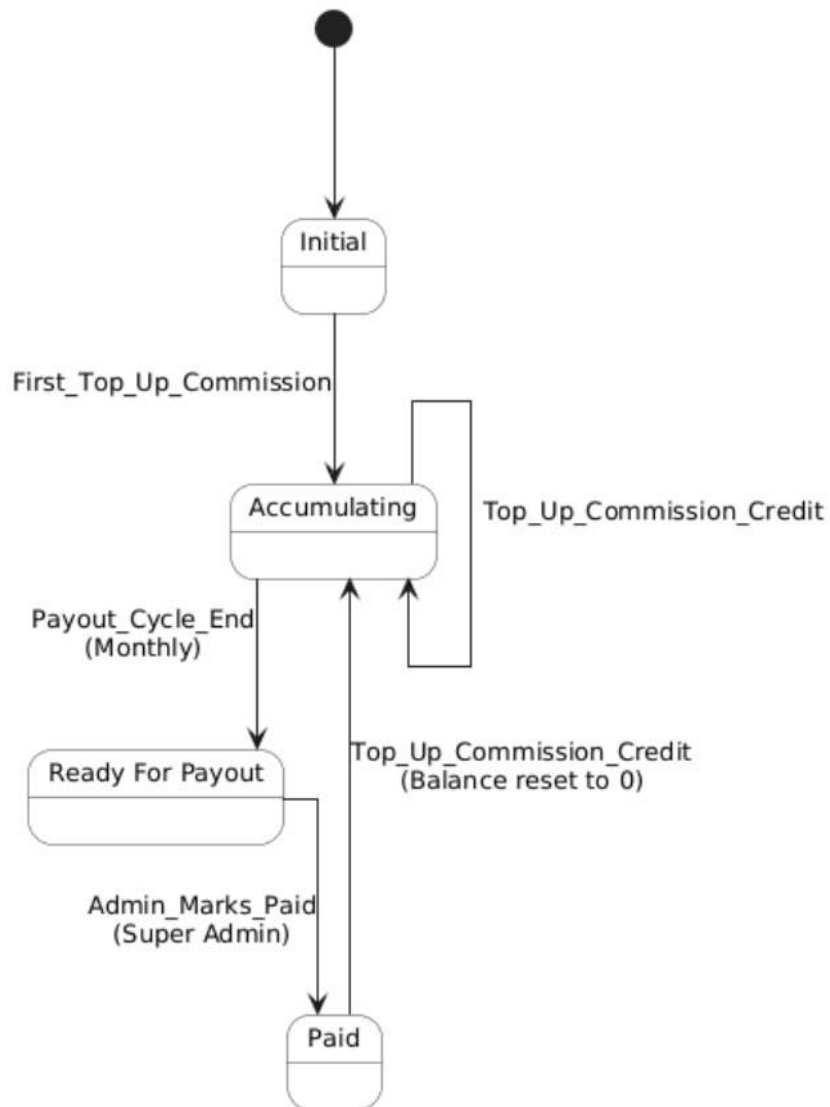


Figure 7 Agent Commision Balance Status

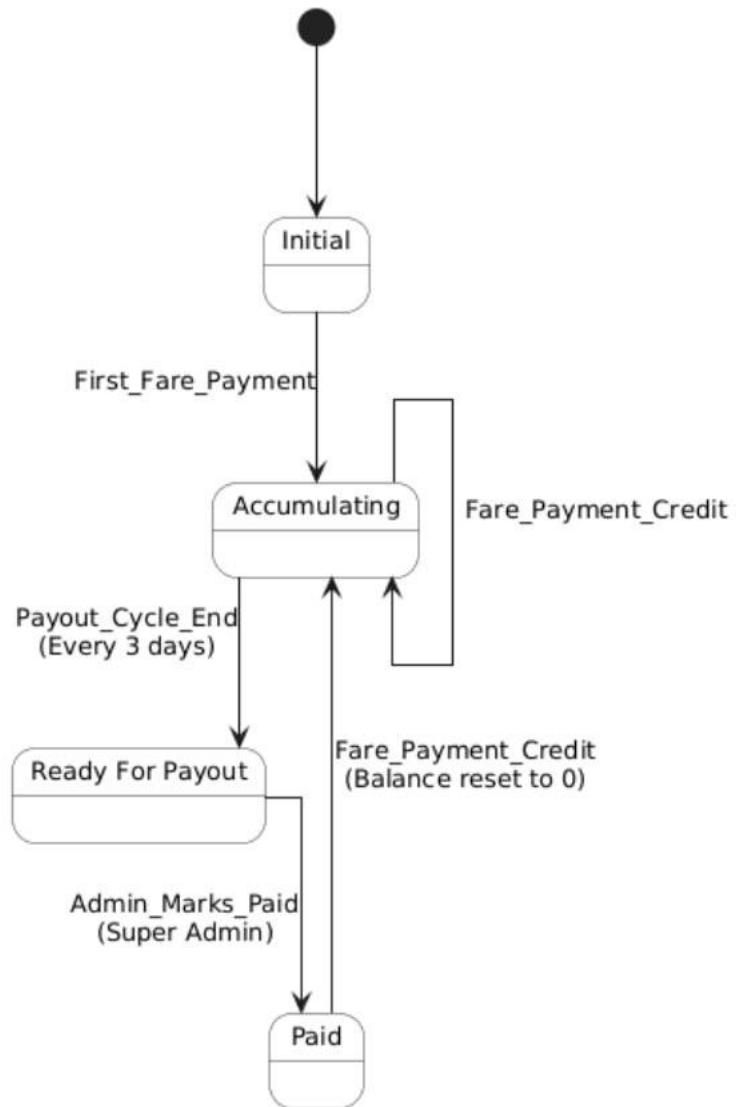


Figure 8 Driver Earning Status

2.3.6 Activity Diagram

The Activity Diagram models the core business process of Fare Payment, illustrating the complete workflow from QR code scanning by the Driver through transaction validation, balance deduction, and confirmation. This diagram captures the interactions between Driver, Passenger, and System components, highlighting decision points and parallel activities for successful payment processing.

Below is the image of the Fare Payment Activity Diagram:

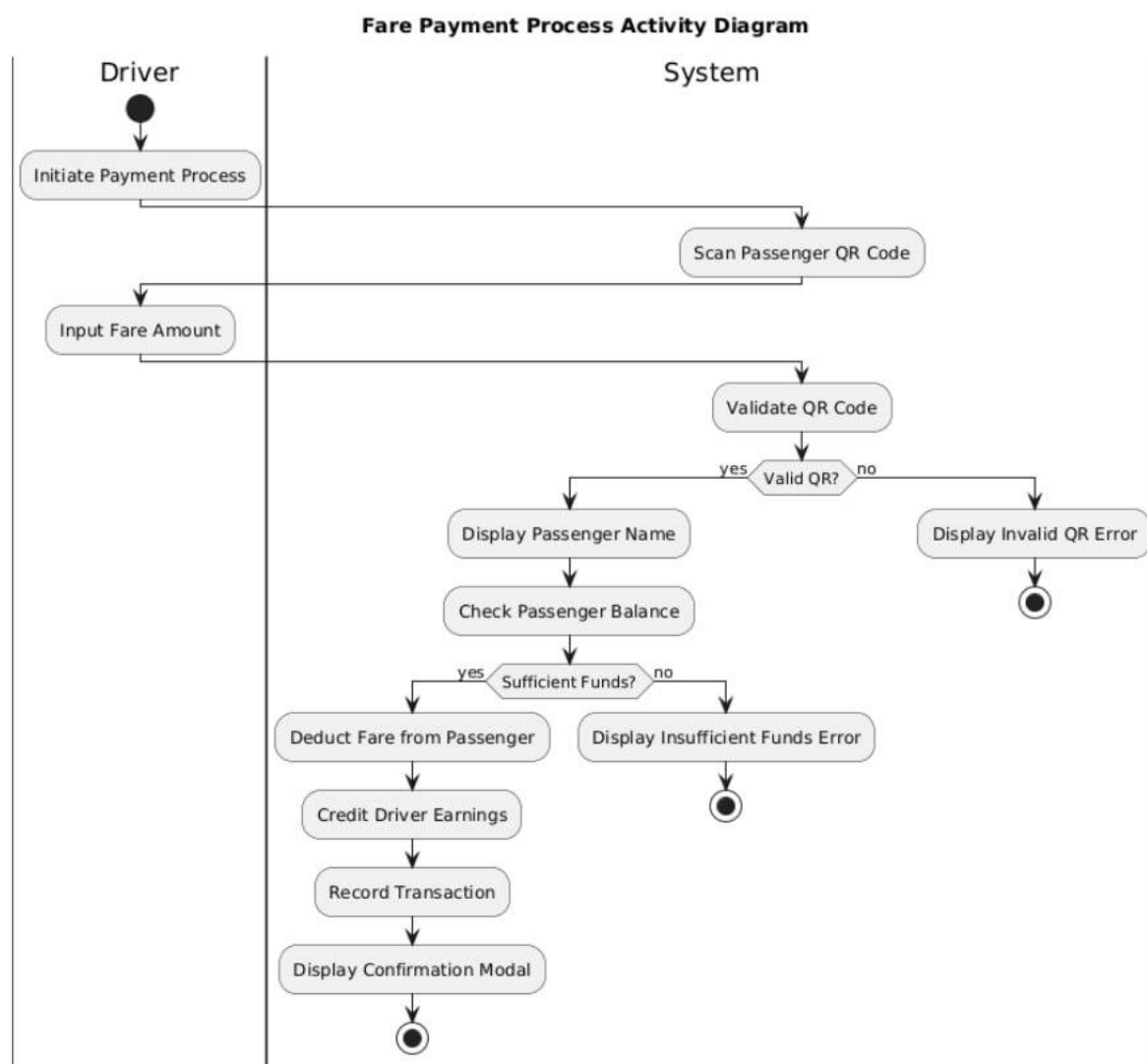


Figure 9 Activity Diagram For Fare Payment process

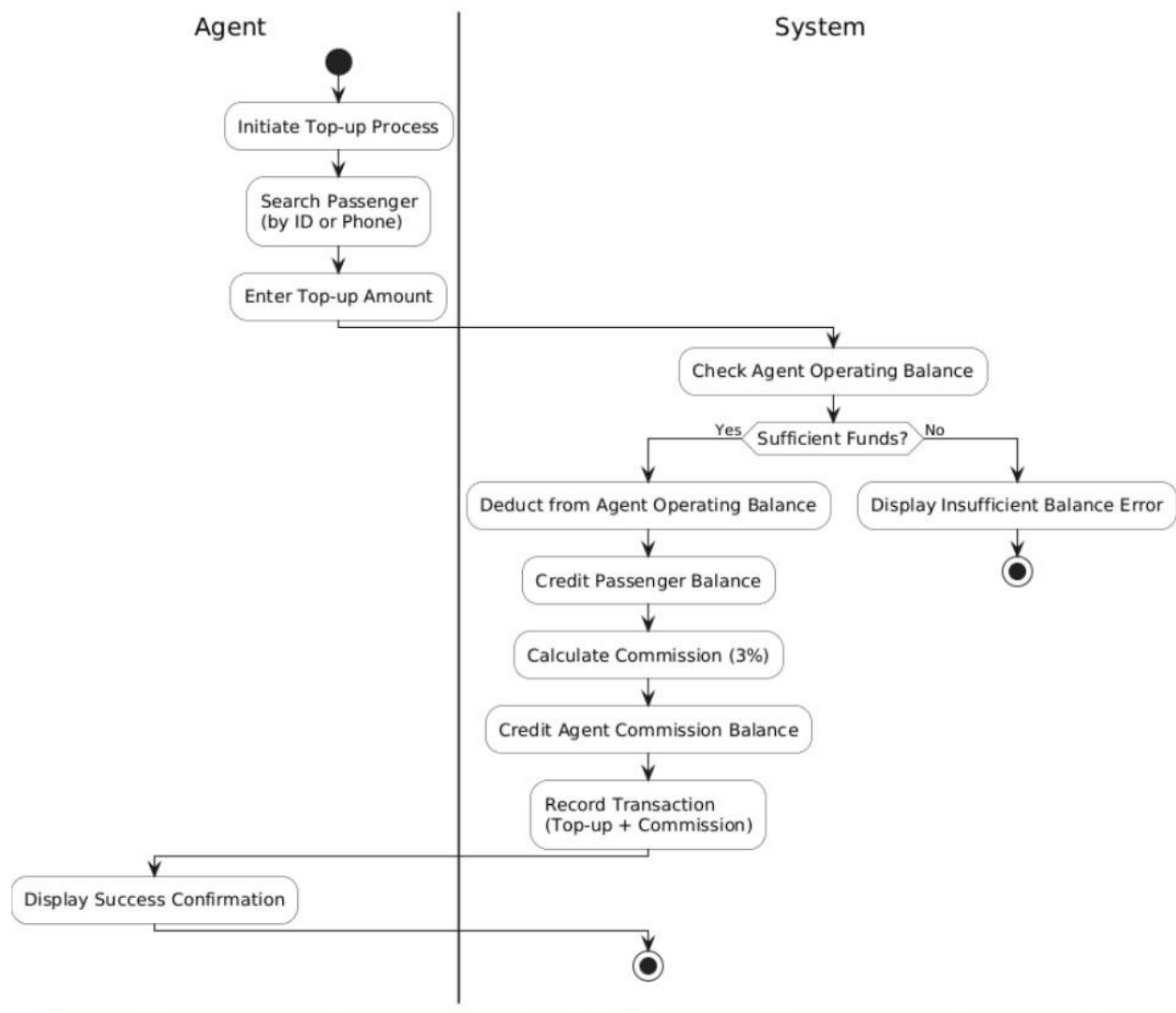


Figure 10 Activity Diagram - Agent Top-up and Commission Process

2.3.7 Analysis Class Model

The Analysis Class Model represents the core conceptual model of the Integrated Digital Payment and Fare Management System, identifying key entities such as User, Passenger, Driver, Agent, Wallet, Transaction, and QRCode along with their attributes, methods, and relationships. This diagram serves as the foundation for the system's object-oriented design and directly evolves into the detailed implementation class diagram.

Below is the image of the Analysis Class Model:

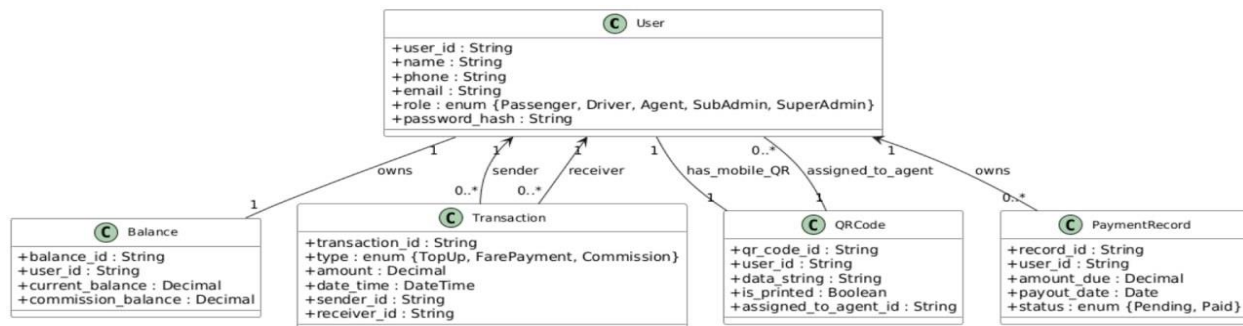


Figure 11 Analysis Class Model

2.3.8 Sequence Diagram

The Sequence Diagram illustrates the dynamic interaction between Driver, Passenger, Wallet Service, and Notification System during the critical Fare Payment process. It shows the message exchanges from QR code scanning through balance validation, fare deduction, driver earnings credit, and transaction confirmation, highlighting the temporal sequence and conditional flows essential for secure payment processing.

Below is the image of the Fare Payment Sequence Diagram:

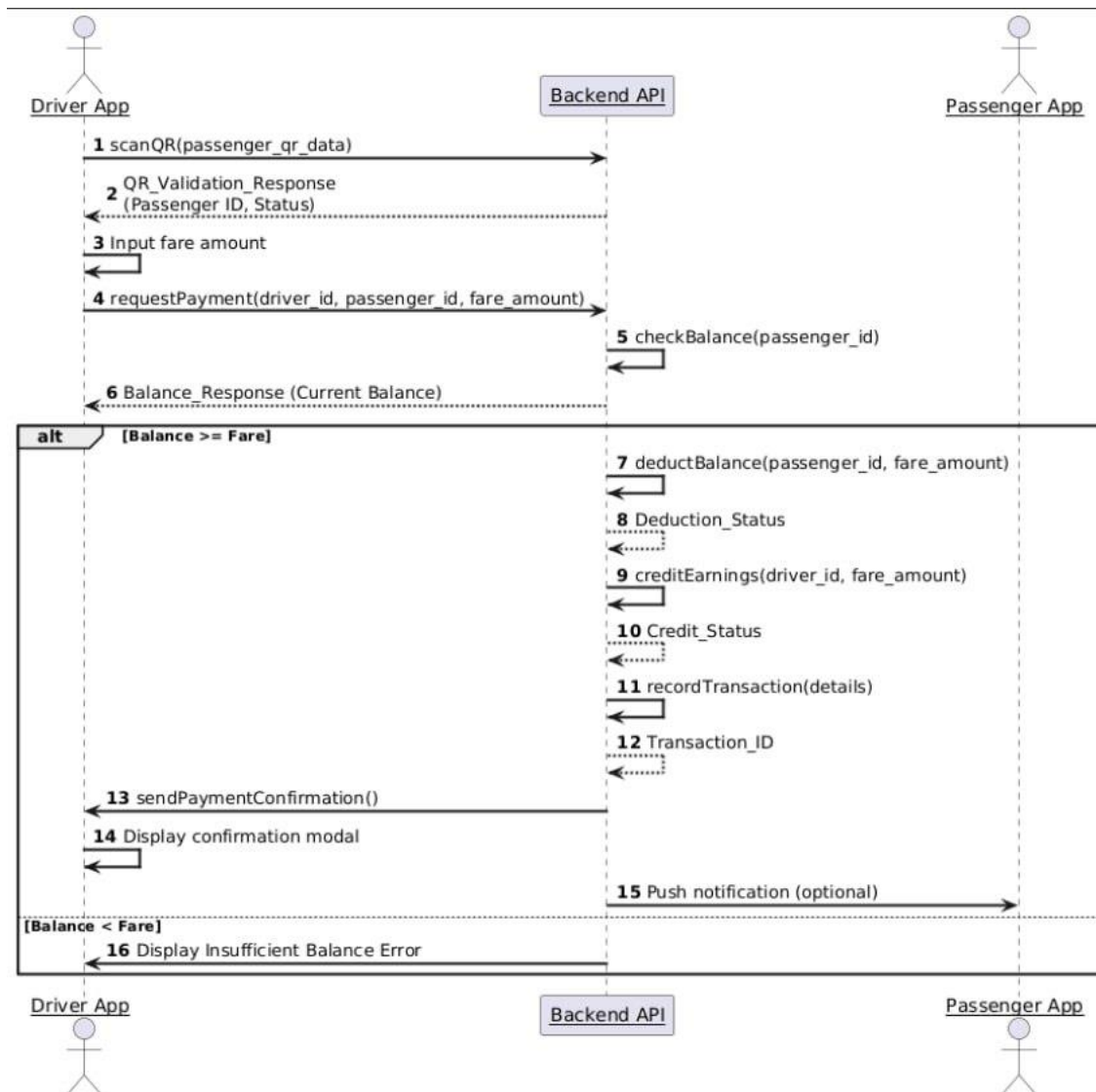


Figure 12 Sequence Diagram For Fare Payment Intraction

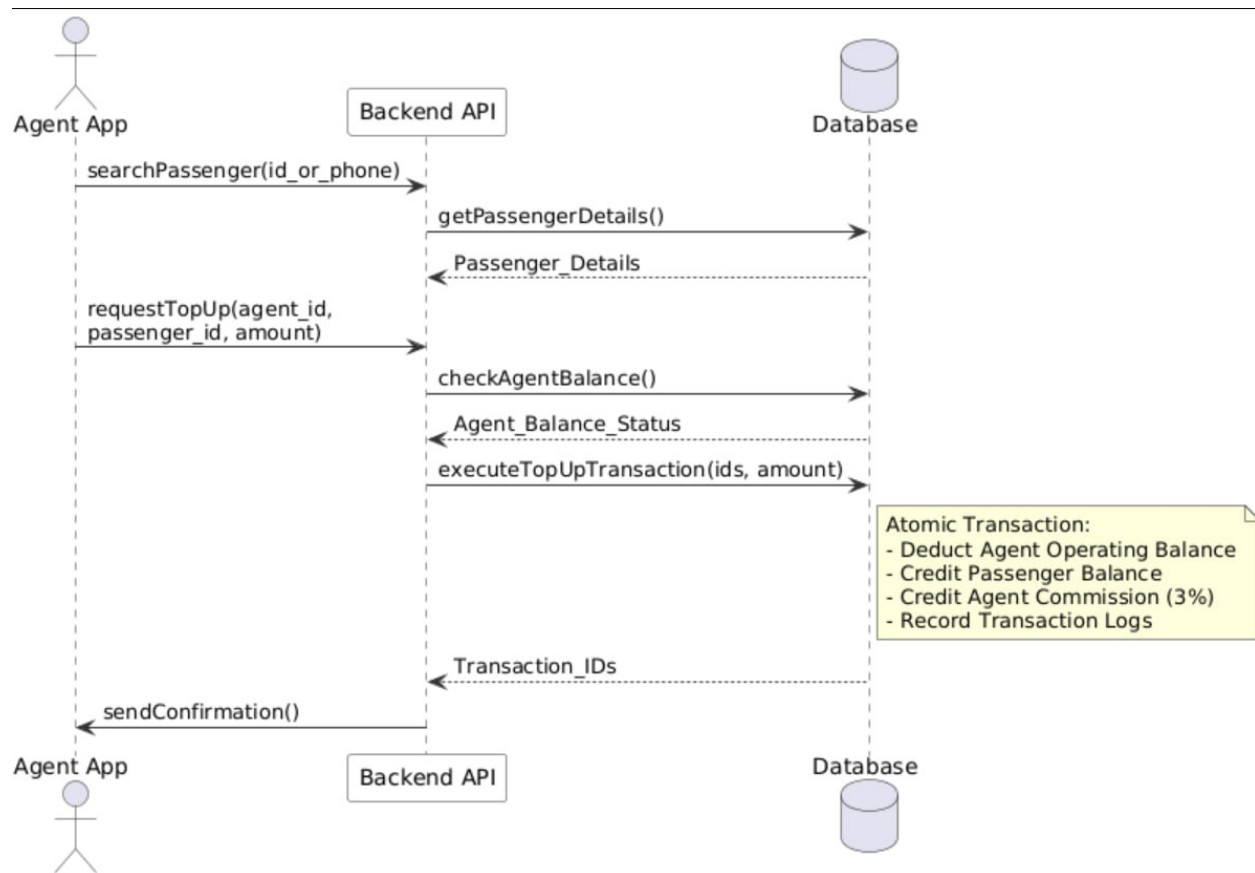


Figure 13 Sequence Diagram Agent Top Up Transaction

2.3.9 Logic Model

The Logic Model defines the core computational algorithm for Agent Top-Up and Commission Calculation, outlining the sequential steps from balance validation through transaction recording. This model captures the business logic for converting agent cash deposits into passenger wallet credits while automatically calculating and allocating the 3% commission to the agent's commission balance.

Below is the image of the Agent Top-Up Logic Model:

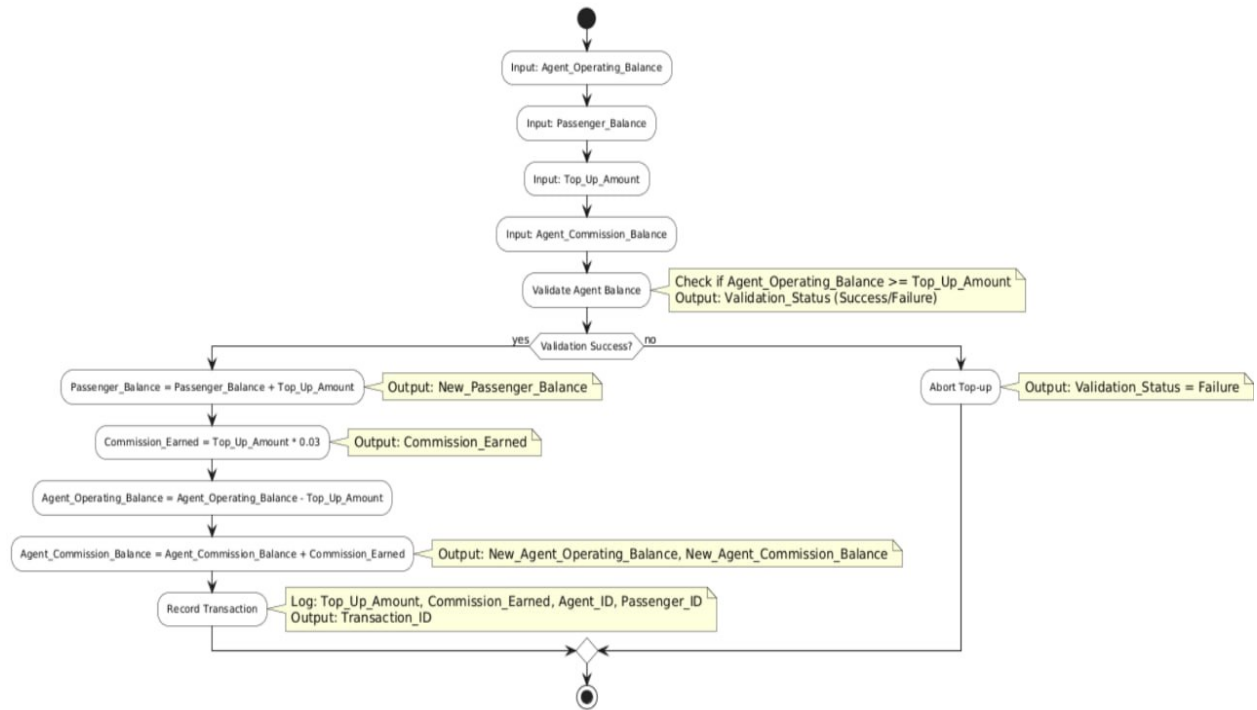


Figure 14 Logic Model For Agent Commission Calculation

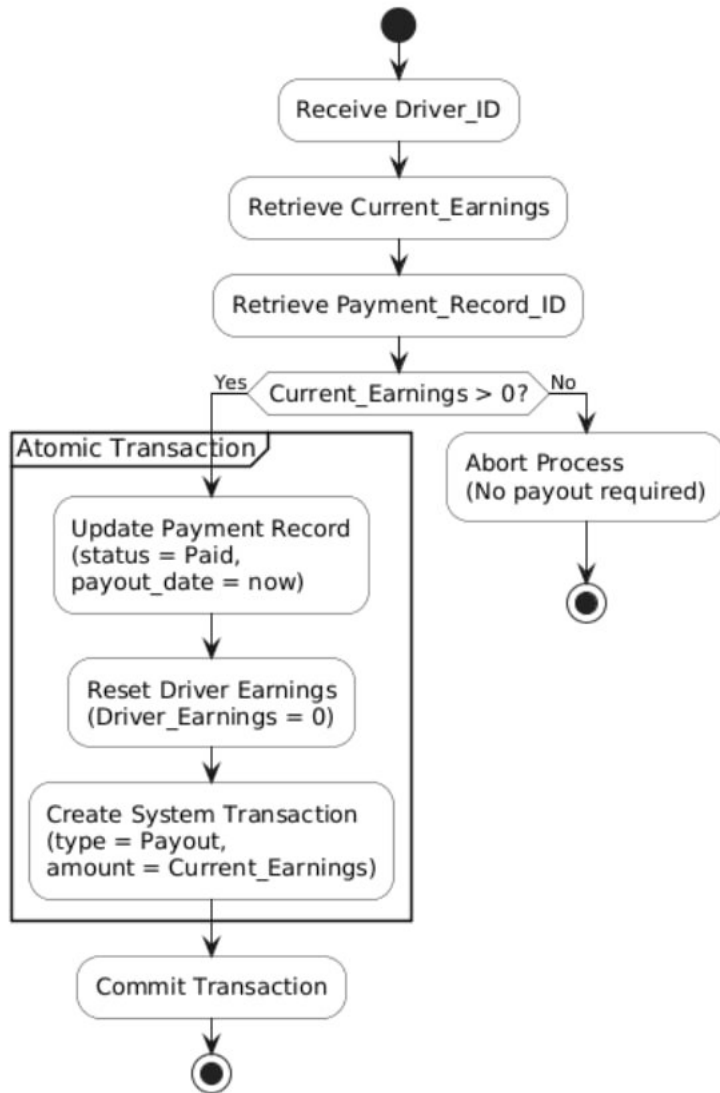


Figure 15 Logic Model Driver Earning Reset

2.4 Non-Functional Requirements

Category	ID	Requirement Description	Priority
Performance	NFR_P_1	The QR code scanning and fare deduction process SHALL complete within few seconds to ensure fast boarding.	Critical
Performance	NFR_P_2	Wallet balance inquiries and transaction history SHALL load quick for all users.	High
Performance	NFR_P_3	The system SHALL support multiple scans during peak hours without degradation.	High
Security	NFR_S_1	All user passwords SHALL be stored using a strong, one-way hashing algorithm (bcrypt with salt).	Critical
Security	NFR_S_2	All communication between Mobile App/Web Admin and Backend API SHALL use HTTPS/TLS 1.3.	Critical
Security	NFR_S_3	The system SHALL implement JWT token-based authentication with 15-minute expiry and refresh tokens.	Critical
Usability	NFR_U_1	Passenger wallet transactions SHALL require OTP verification for amounts above certain amount.	Medium
Usability	NFR_U_2	Web Admin Portal SHALL be responsive on desktop (1024px+), tablet (768px+), and major browsers (Chrome, Firefox, Safari).	Medium
Reliability	NFR_R_1	The system SHALL maintain 99% uptime.	Critical
Reliability	NFR_R_2	All transactions SHALL be atomic - either fully committed or fully rolled back on failure.	Critical
Maintainability	NFR_M_1	Codebase SHALL follow RESTful API standards, ESLint, and modular	Medium

Node.js architecture with Multer for file handling.

Table 8 Non functional Requirment

2.5 System Requirments

2.5.1 Hardware Requirements

Component	Requirement	Rationale
Backend Server	Laptop/Desktop: 2 vCPUs, 4GB RAM, 20GB free (for testing)	Handles demo with 10-20 test users and sample transactions.
Database	Dedicated or managed PostgreSQL instance	To ensure data integrity, high availability, and fast query response times
Mobile Devices	Android (v . +) or iOS (v +) with a working camera for QR scanning.	Runs the Flutter app for QR scanning and wallet demo.
Admin Devices	Any laptop/desktop with Chrome/Firefox	Access to web admin dashboard for demo presentations.
Network	0.5Mbps + upload/download with <200ms latency	Ensures fast QR scan confirmation and real-time transaction updates.

Table 9 Hardware Requirment

2.5.2 Software Requirements

Component	Requirement	Rationale
Operating System	Ubunt 20.04 LTS or Windows 10/11	Stable environment for Node.js development and local PostgreSQL.
Backend Runtime	Node.js 18+ LTS, Express 4.18+	Required to run the REST API, JWT auth, and Multer file handling.

Database	PostgreSQL 13+	Stores users, wallets, transactions, and QR codes with proper indexing.
Web Browser	Chrome 100+, Firefox 90+, Edge 100+	Required for responsive Web Admin Portal and dashboard viewing.
Mobile OS	Android 8.0+ (API 26), iOS 13+	Minimum for React Native/Flutter app with camera QR scanning.
Development Tools	npm 9+ or Flutter 3+	Builds mobile apps and manages dependencies for class project demo.
Payment Gateway	Chapa API (sandbox mode)	Handles test wallet top-ups via mobile money integration.

Table 10 Software Requirement

2.6 Key Abstraction with CRC Analysis

The Key Abstraction with CRC Analysis identifies core concepts and objects essential to the Fare Payment system, documenting their responsibilities (R), collaborations (C), and class relationships through CRC cards. This analysis captures key entities like Passenger, Driver, QRCode, Wallet, and Transaction, illustrating how they interact to support registration, top-up, scanning, and reporting functionalities.

2.6.1 Conceptual Modeling: Class Diagram

We have identified many diagram to represent our proposal below we mentioned class activity sequence, state, analysis and logical diagrams.

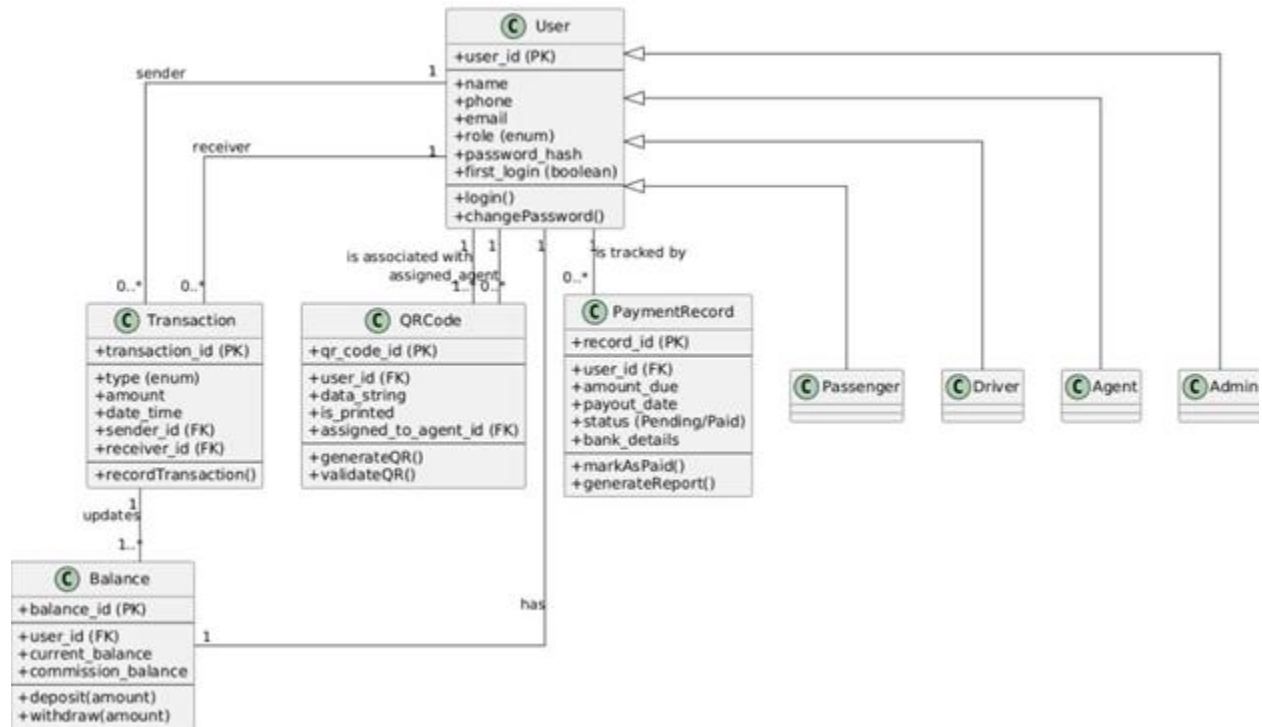


Figure 16 Class Diagram

2.6.2 Identifying Change Cases

Change cases identify potential future modifications to the system requirements, assessing their likelihood of occurrence and impact on the current design. This analysis helps evaluate system flexibility and maintenance costs for anticipated evolution scenarios such as new payment integrations, pricing models, or operational changes.

Change Case ID	Description	Likelihood	Impact	Affected Components
CC_01	Add support for new payment gateway (HelloCash alongside Chapa)	Medium	High	Wallet, Passenger Module, F_PASS_5
CC_02	Support multiple fare types per route (fixed vs distance-based)	High	Medium	All UI components
CC_03	Add multi-language support (Amharic/English)	Medium	Low	All UI components
CC_04	Implement driver rating system	Low	High	Driver, Passenger modules
CC_05	Dynamic Commission Rate- Change Agent commission rate via Web Admin Portal	Medium	Medium	F_AGNT_5, F_SADM_1
CC_06	Automated Bank Payout- Bank API for payouts	Low	Critical	F_ADM_1, F_ADM_2
CC_07	Fare Zones/ Variable Pricing- GPS/route-based fares	High	High	F_DRIV_3
CC_08	Driver Withdrawal Request - Partial payouts before cycle	Medium	Medium	F_DRIV_4, F_ADM_1

Table 11 Change Case

2.6.3 User Interface Prototype

The User Interface Prototype presents interactive high-fidelity mockups of the Fare Payment system, showcasing refined screens for Passenger QR generation, Driver scanning/validation, payment confirmation, and wallet management flows. This prototype simulates real user interactions across mobile apps, enabling stakeholder validation of usability, visual design, and

navigation before final implementation.

Below is the image of the Fare Payment User Interface Prototype:

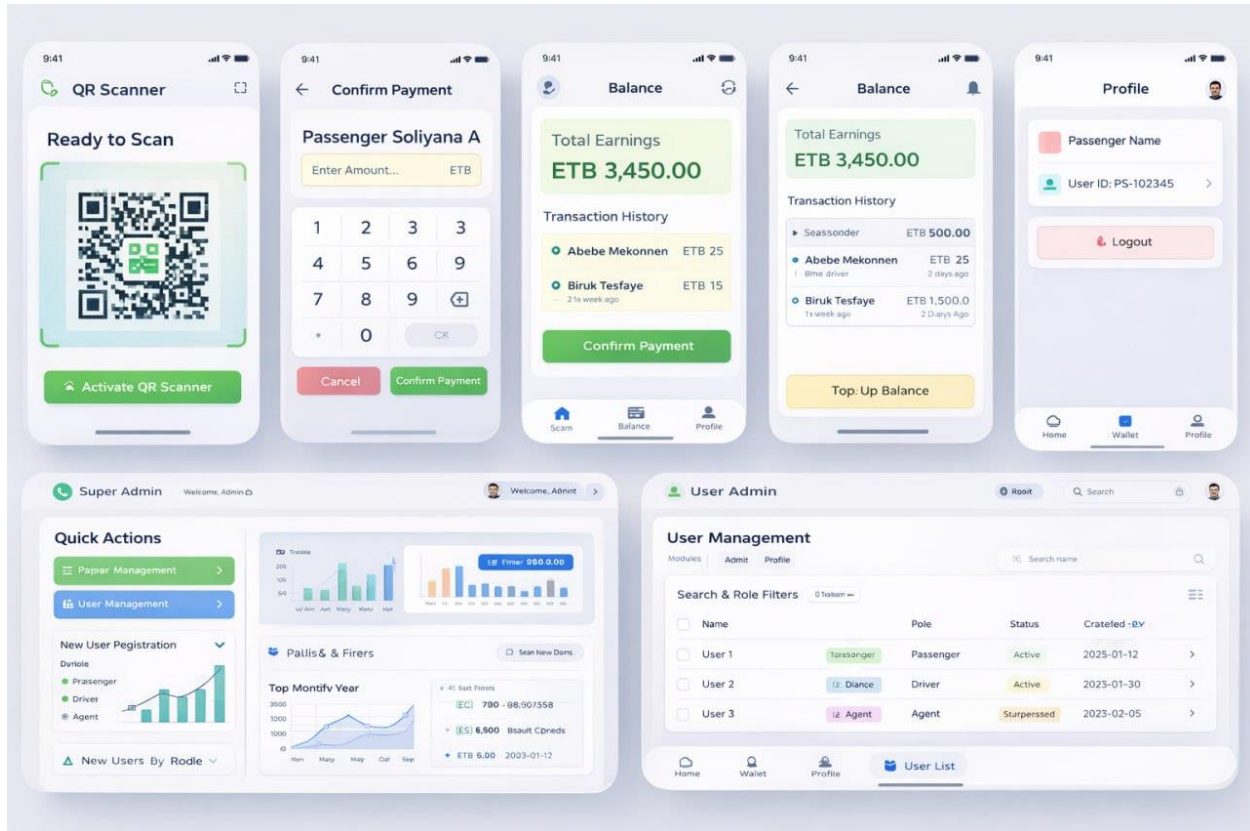


Figure 17 User Interface Prototype

Chapter 3: System Design

System Overview and Purpose

Integrated Digital Payment and Fare Management is a digital payment system developed to improve shared transportation services such as taxis and buses by replacing traditional cash-based fare collection with a secure QR code-based digital wallet system.

The system is designed to address three major challenges commonly found in shared transport operations:

1. **Cash Inefficiency** – Eliminating delays caused by cash handling and lack of change.
2. **Lack of Accountability** – Providing transparent and traceable digital transaction records for all stakeholders.
3. **Limited Accessibility** – Extending digital payment services to users who do not own smartphones.

By addressing these challenges, Integrated Digital Payment and Fare Management aims to improve efficiency, transparency, and inclusiveness within the transport payment ecosystem.

Design Goal: Performance

Integrated Digital Payment and Fare Management is designed to support fast and reliable operations in high-traffic transport environments.

The system uses optimized QR code scanning to ensure quick fare processing, allowing passengers to board vehicles with minimal delay. A Node.js and Express-based backend, integrated with a PostgreSQL database, ensures that fare payments and balance updates are processed in near real time.

The system follows a lightweight RESTful API design to reduce network overhead, making it suitable for mobile users with limited data access while maintaining responsive system performance.

Design Goal: Dependability

Reliability and data integrity are critical for systems that manage financial transactions.

All payment and top-up operations in Integrated Digital Payment and Fare Management are handled as atomic transactions, ensuring consistency and preventing partial or lost updates. The system supports a multi-balance structure where agent commissions are calculated and recorded automatically during top-up operations.

Role-based authentication using JSON Web Tokens (JWT) restricts access to system features based on user roles, reducing the risk of unauthorized actions. Additionally, the Super Admin dashboard provides complete transaction logs and supports exporting records in Excel format for auditing and verification purposes.

Design Goal: End-User Experience and Accessibility

Integrated Digital Payment and Fare Management is designed to be easy to use for individuals with varying levels of technical experience.

The system supports three main user roles:

- **Passengers**, who use a simple QR code interface to pay fares.
- **Drivers**, who use a scanning interface to collect payments efficiently.
- **Agents**, who manage user registration and account top-ups through a simplified workflow.

To support users without smartphones, Integrated Digital Payment and Fare Management allows the use of printed QR codes. These users can register through agents and receive physical QR cards or stickers, ensuring inclusion without requiring mobile devices. For security purposes, drivers and agents are required to change their passwords upon first login.

Design Goal: Scalability and Security

The system architecture is designed to support future expansion and increased usage.

Integrated Digital Payment and Fare Management uses a decoupled architecture where the backend API operates independently from the mobile application and web-based admin interface. This separation allows each component to scale independently based on system demand.

For financial security, mobile deposits are handled through Chapa, a trusted third-party payment gateway, ensuring secure processing of sensitive payment information. Manual payout controls managed by the Super Admin provide an additional layer of verification, reducing the risk of automated financial errors while maintaining accurate digital records.

3.1 Architectural Design

The Integrated Digital Payment and Fare Management System is designed using a Layered Architecture (also known as multi-tier architecture) to achieve a clear separation of concerns, improve maintainability, and support scalability. The system is logically organized into four distinct layers: Presentation, Application, Data, and External Services. This architectural approach directly supports the system's non-functional requirements for security (NFR_S_2) and maintainability (NFR_M_1) by decoupling user interface components from business logic and data management.

At the heart of the system is a robust, stateless Application Tier implemented with Node.js and Express, which leverages non-blocking I/O to efficiently handle high volumes of concurrent transaction requests during peak periods, thereby satisfying performance requirements (NFR_P_3).

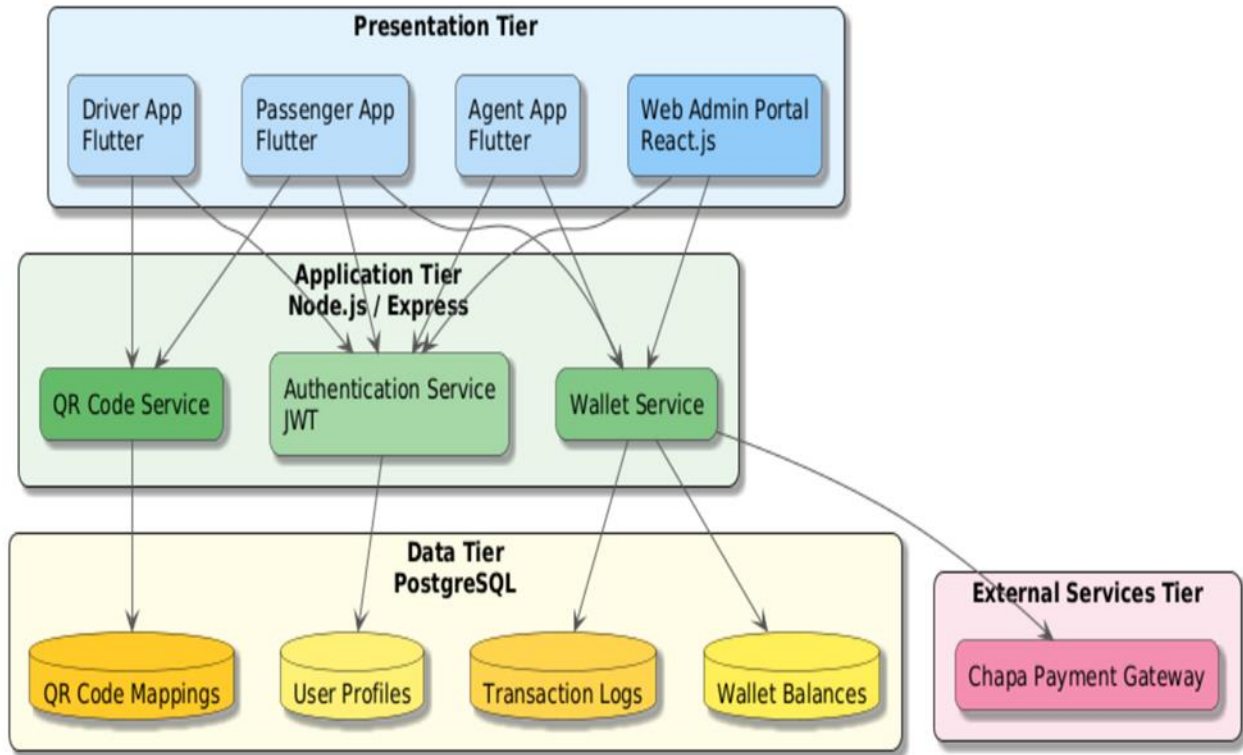


Figure 18 Architectural Design

3.1.1 Component Modeling

The Application Tier is structured into a set of independent, modular components, each dedicated to a specific set of functionalities. This component-based design enables parallel development and testing while supporting seamless future enhancements and system evolution. The key components and their respective responsibilities are outlined below:

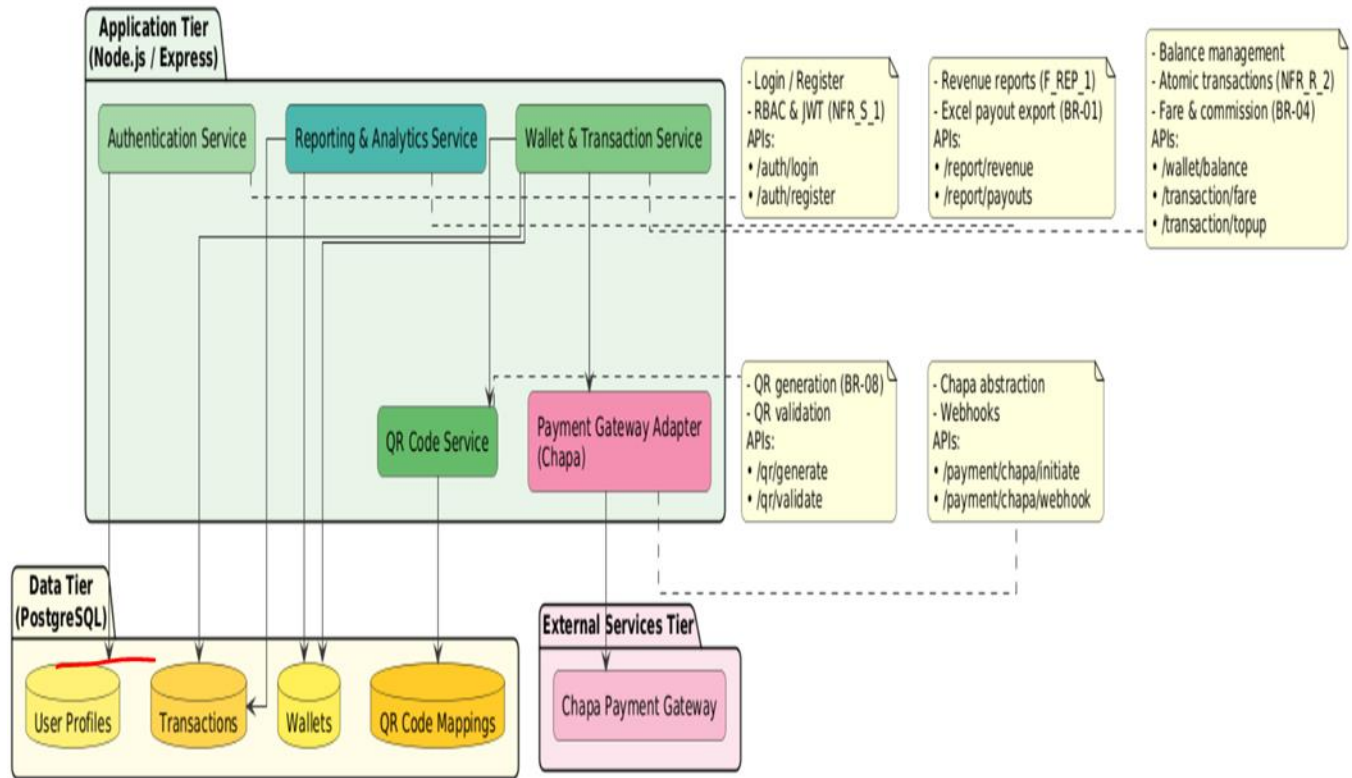


Figure 19 Component Modeling

This modular design ensures that updates or changes to external dependencies—such as replacing Chapa with an alternative payment gateway—are confined to the Payment Gateway Adapter, thereby minimizing any impact on the core business logic within the Wallet & Transaction Service.

3.1.2 Deployment Modeling

The system is designed for a modern, cloud-native deployment environment, leveraging platforms such as Koyeb (as referenced in Section 1.4.3) to ensure high availability and scalability. The deployment architecture consists of three primary physical layers: Client Nodes, the Application Cluster, and the Data Store, complemented by external services.

1. Client Nodes

- **Mobile Devices:** Host the Flutter-based Passenger, Driver, and Agent applications, communicating securely with the Application Cluster over mobile networks.
- **Web Browser:** Hosts the React.js Web Admin Portal, accessed by Super Admins and Sub Admins.

2. Application Cluster (Cloud Platform)

- Node.js API Instances: Multiple, load-balanced instances of the Node.js/Express Application Tier that process incoming requests, execute business logic, and provide system resilience, ensuring 99% uptime (NFR_R_1).
- Load Balancer / API Gateway: Manages traffic distribution across API instances and handles TLS termination for secure communication.

3. Data Store (Aiven / Managed PostgreSQL)

- PostgreSQL Database: A dedicated, managed database instance or cluster that acts as the single source of truth for all persistent data. Using a managed service provides automated backups, failover, and performance optimization.

4. External Services

- Chapa Payment Gateway: A secure, third-party service accessed by the Application Cluster to process financial transactions.

The deployment strategy prioritizes horizontal scaling of the stateless Node.js API instances to efficiently handle fluctuating demand during peak transport hours (NFR_P_3). Critical services, such as the Wallet & Transaction Service, rely on the Data Store's transactional integrity to ensure atomicity and reliability of all financial operations (NFR_R_2).

Below we have attached an image of the Development Model.

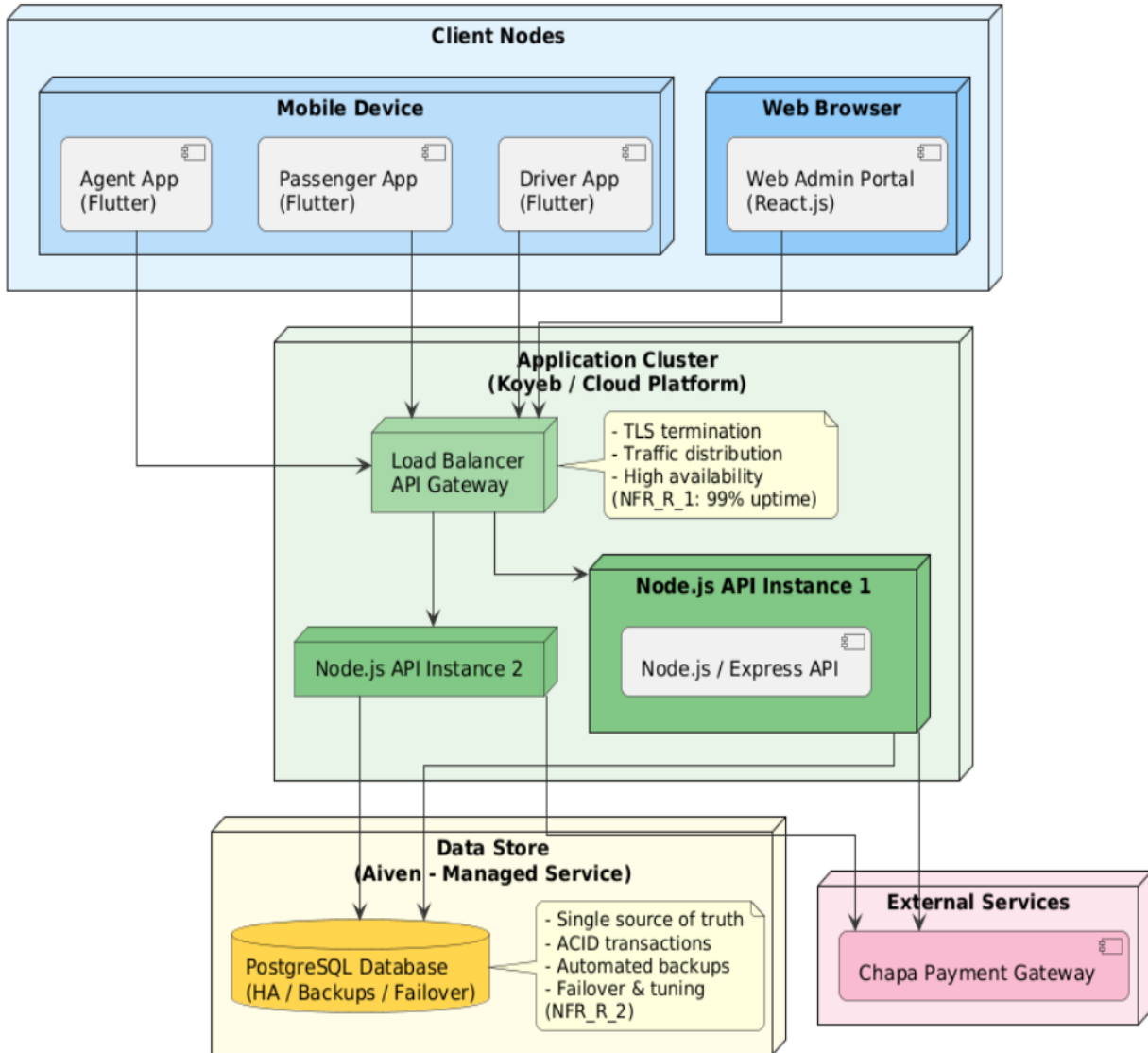


Figure 20 Deployment Modeling

3.2 Detailed Design

The detailed design phase translates the system requirements and high-level architecture into precise, implementable components. It defines how each part of the Integrated Digital Payment and Fare Management system operates internally, including class structures, database models, service logic, and interaction between system layers.

This design focuses on clear separation of concerns, where controllers handle API requests, services manage business logic, and persistence models ensure reliable data storage. Role-based access control, transaction workflows, QR-based payments, and commission calculations are all designed at this level to ensure correctness, security, and scalability.

3.2.1 Design Class Model

The Design Class Model illustrates the internal structure of the Integrated Digital Payment and Fare Management system by identifying the main classes, their attributes, methods, and relationships. It translates the system requirements into a technical design that guides implementation. The model focuses on core domain entities such as users, wallets, transactions, QR codes, and payments, showing how responsibilities are distributed across the system and how different components interact to support system functionality. Below we have put the class model of the design.

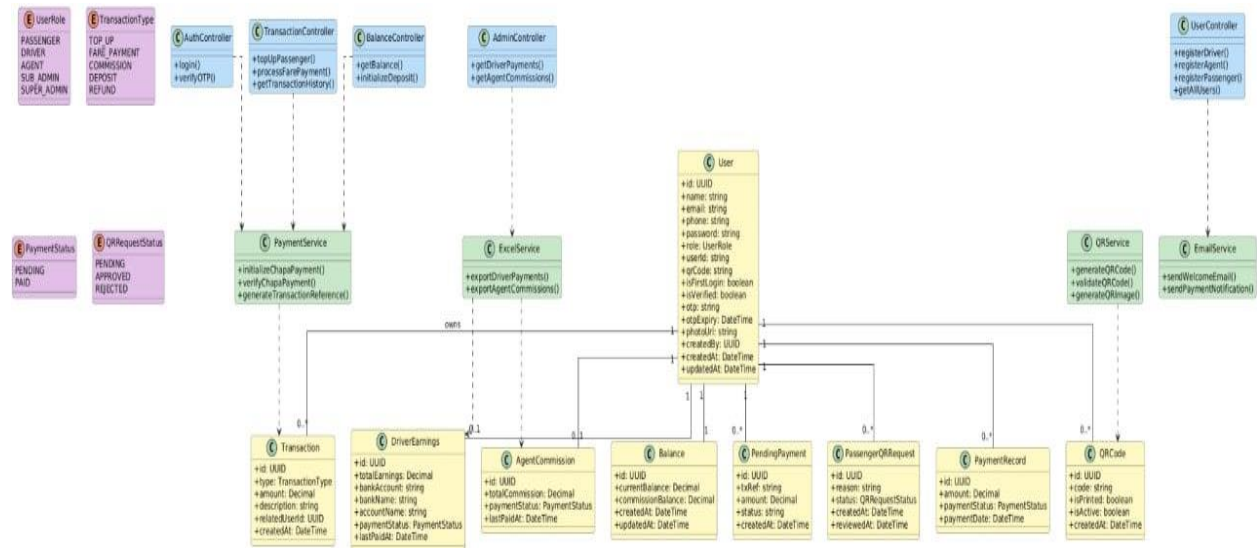


Figure 21 Design Class Model

3.2.2 Persistence Model – Introduction

The Persistence Model describes how the system’s data is stored and managed within the database. It defines the main data entities, their attributes, and relationships, ensuring data consistency and integrity across the Integrated Digital Payment and Fare Management system. This model supports core operations such as user management, transactions, QR code handling, and payment records by providing a structured representation of persistent data. Below we have attached the image of persistence class model.

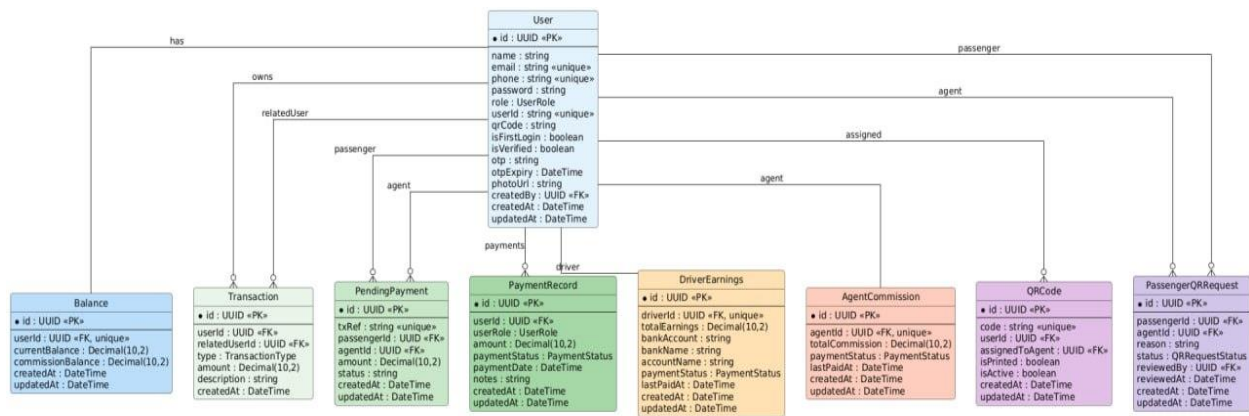


Figure 22 Persistence Class Model

3.3 User Interface Design

Landing Page

The Landing Page serves as the entry point for all Integrated Digital Payment and Fare Management users, providing quick access to login, registration, and system information. It presents a clear and user-friendly interface to guide passengers, drivers, and agents to their respective functionalities. The layout is designed to be intuitive, ensuring smooth navigation and immediate access to core system features.

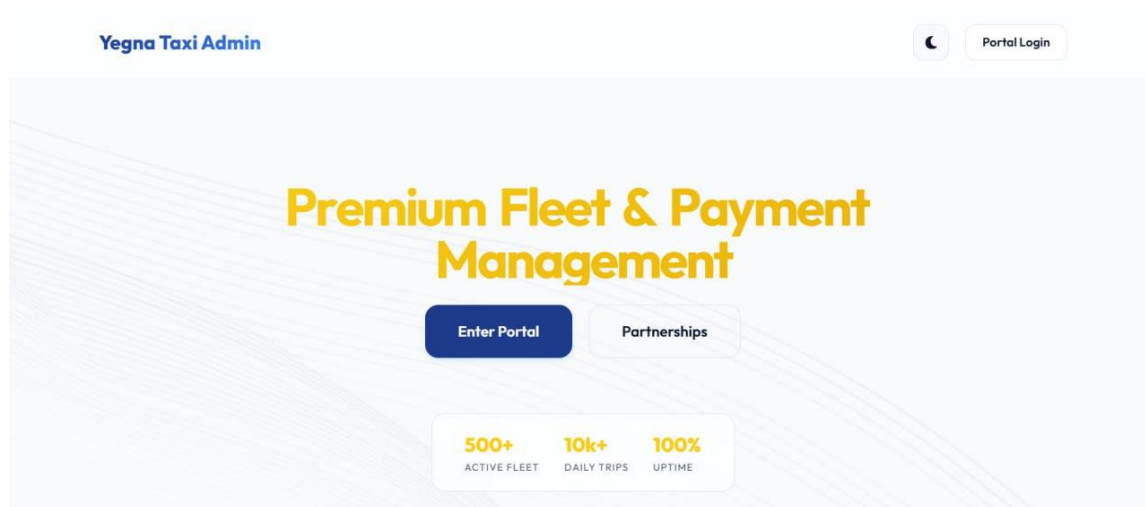


Figure 23 User Interface-Landing Page

Wallet Top-Up User Interface

The wallet top-up interface allows passengers and agents to securely add funds to their accounts through the integrated Chapa payment system. Below there is the image of agent and passenger topping up interface respectively.

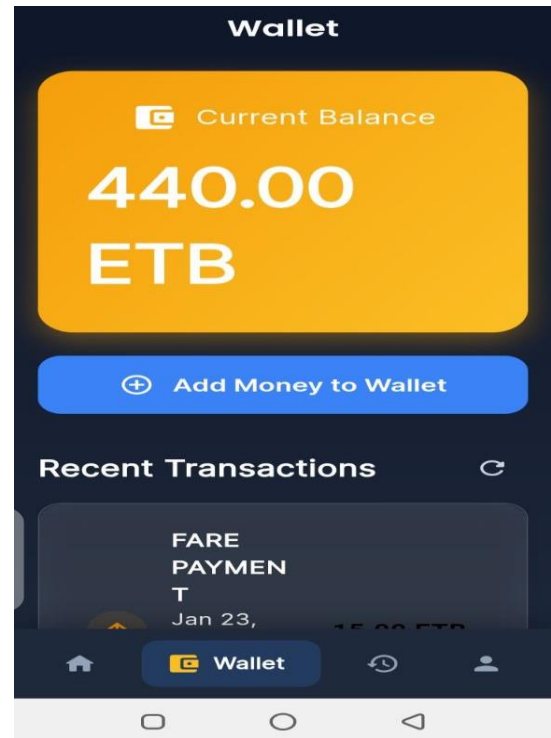
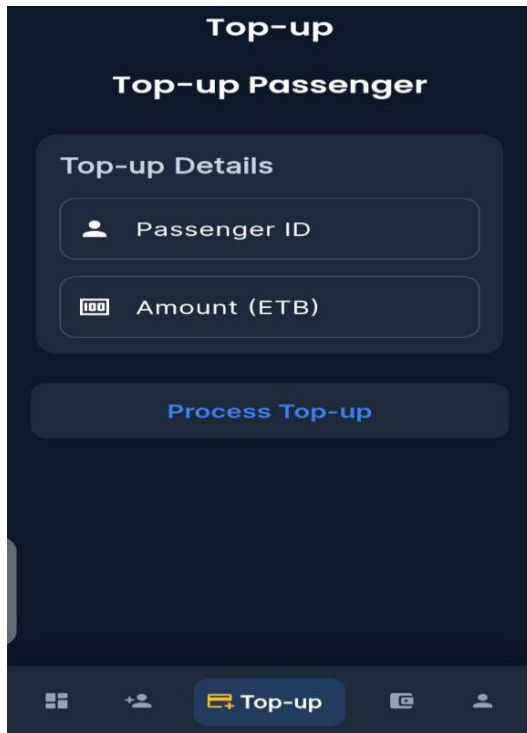


Figure 24 User Interface - wallet Top-up

Dashboard Overview

The dashboard provides a quick overview of the user's wallet balance for passengers, agents, and drivers. For drivers, it also includes QR scanning and fare collection options, while agents and passengers can view balances and initiate top-ups. Below there are screenshots attached to the passenger, driver and agent dashboard respectively.

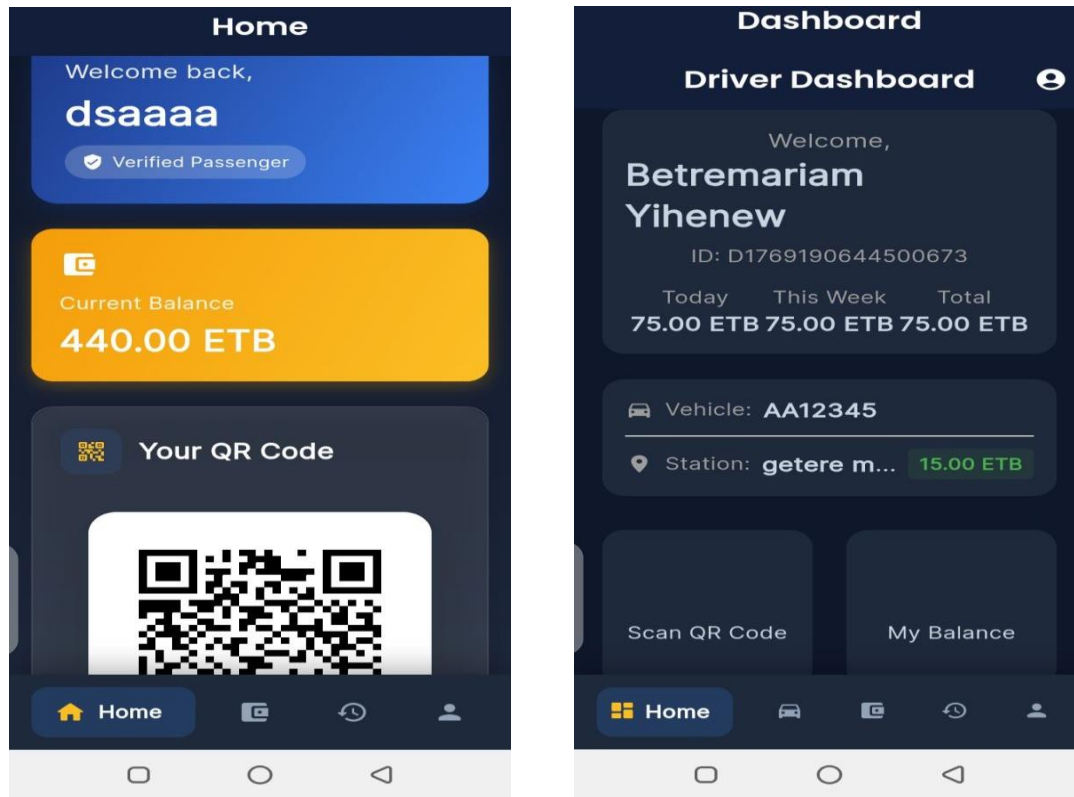


Figure 25 User Interface - Passenger and Driver Dashboards

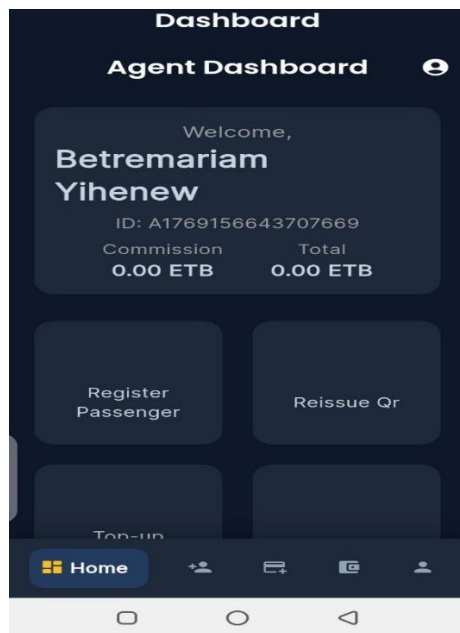


Figure 26 User Interface - Agent Dashboard

Transaction Overview

The dashboard allows passengers, agents, and drivers to view their recent transactions. Users can track top-ups, fare payments, and commissions in a clear, organized format for easy reference.

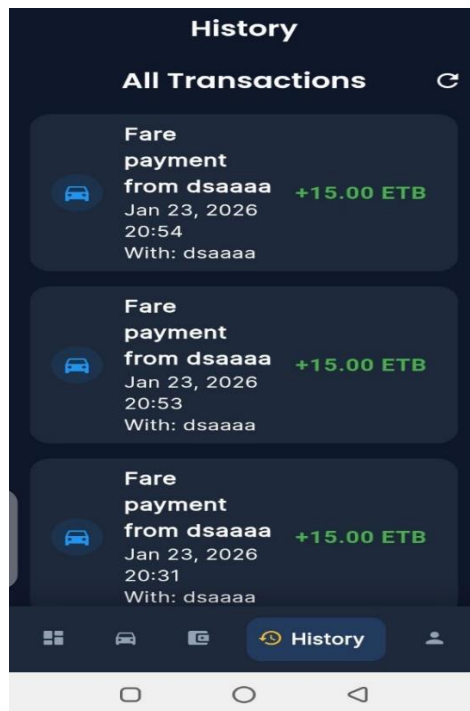


Figure 27 User Interface - Transaction History

Super Admin Payments Management Page

The Super Admin can view all drivers and agents, track their earnings and commissions, and process payments to multiple users at once. This page streamlines the management of driver and agent payments, ensuring accurate and timely settlements across the system.

NAME	USER ID	PHONE	TOTAL EARNINGS	BANK ACCOUNT	STATUS	LAST PAID
Betremariam Yiheneu	D1766639027693722	0985858585	0.00 ETB	CBE 1000237286506	PENDING	-
Betremariam Yiheneu	D1769190644500673	0985246234	75.00 ETB	CBE 1000237286508	PENDING	-
demeke	D1769158734449760	0923393685	0.00 ETB	cbe 1000237286506	PAID	23/01/2026

Figure 28 Web Admin - Driver Payment Management

NAME	USER ID	PHONE	TOTAL COMMISSION	STATUS	LAST PAID
Betremariam Yiheneu	A1769156643707669	0985246232	0.00 ETB	PENDING	-
efrem	A1764990757436578	0985246218	0.00 ETB	PENDING	-
estif	A1764996966077696	0985246222	0.00 ETB	PENDING	-
nazi	A1768939651134431	0985246227	0.00 ETB	PENDING	-

Figure 29 Web Admin - Agent Commission Management

Super Admin Analytics Page

The Super Admin can access an analytics board that provides a comprehensive overview of transactions, user activity, and key system records. This feature enables the admin to monitor system performance, track financial flows, and make informed decisions based on real-time data.

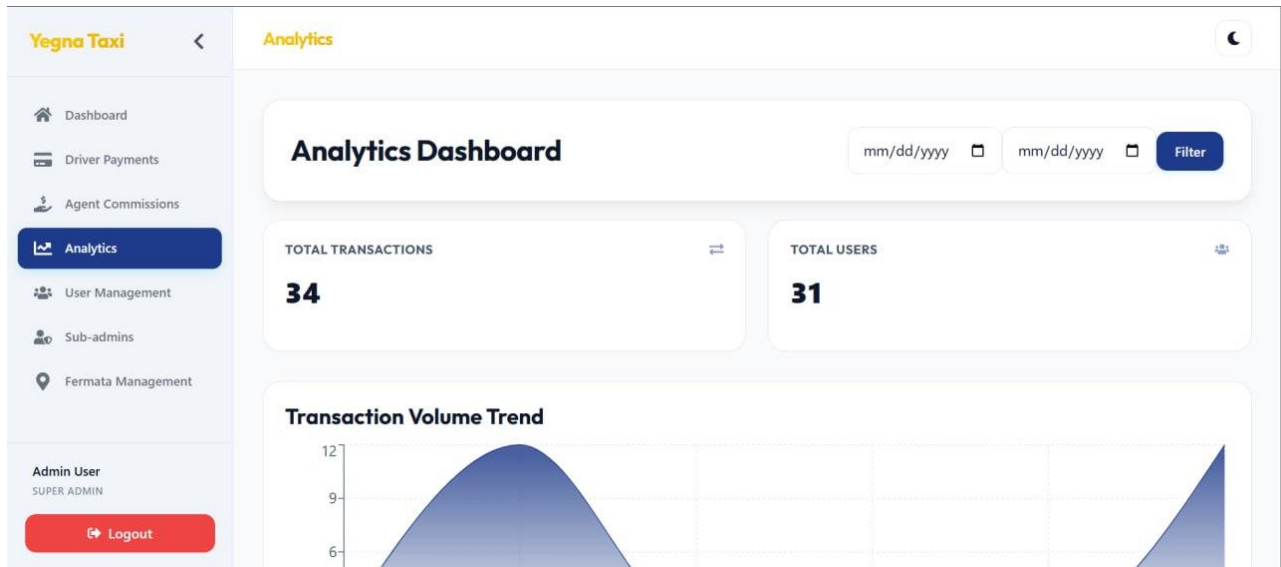


Figure 30 Web Admin - Analytics Dashboard

QR Code Management for Non-Smartphone Passengers

Passengers without smartphones are provided with physical QR codes, which are issued and assigned through agents. The system also allows Sub Admins to manage QR re-issue requests, ensuring that lost or damaged QR codes are replaced efficiently. This process guarantees that all passengers, regardless of device availability, can use the Integrated Digital Payment and Fare Management system for fare payments.

Yegna Taxi < QR Code Reissue

Dashboard
Register User
QR Code Assignment
QR Code Reissue
Analytics

Sub Admin User
SUB ADMIN
Logout

QR Re-issue Requests by Agent

Select Agent
Betremariam Yihenev — A1769156643707669 (0985246232) ▼

Mark All as Processed Download All QRs (ZIP) Submit 0 as Processed

NAME	USER ID	PHONE	QR CODE	DOWNLOAD	STATUS
abebe	P176915637598753	0985246231		Download	Save Mark

Figure 31 Web Admin - QR Re-issue Requests

Agent and Driver Registration

The system allows Sub Admins to register new agents and drivers through dedicated registration forms. During registration, essential details such as name, phone number, email, and role-specific information are captured. For drivers, bank account details can also be added to facilitate earnings settlement. Registered users are then assigned unique IDs, and login credentials are generated securely, enabling them to access the Integrated Digital Payment and Fare Management system according to their roles.

The screenshot shows a web application interface for 'Yegna Taxi'. On the left is a sidebar with a navigation menu containing 'Dashboard', 'Register User' (highlighted in blue), 'QR Code Assignment', 'QR Code Reissue', and 'Analytics'. Below the menu, it identifies the user as 'Sub Admin User' with the role 'SUB ADMIN' and a red 'Logout' button. The main content area is titled 'Register User' and contains a form with the following fields: 'Role' (a dropdown menu with 'Driver' selected), 'Name' (a text input with placeholder 'Enter full name'), 'Phone' (a text input with placeholder 'e.g. 0911001122'), 'Email (Optional)' (a text input with placeholder 'e.g. user@example.com'), and 'Bank Account (Optional)' (a text input with placeholder 'Enter bank account number').

Figure 32 Web Admin - User Registration Form

User and Sub Admin Management

The system provides comprehensive user management functionality for administrators. Super Admins and Sub Admins can view, filter, and manage all registered users, including passengers, drivers, and agents. They can update user details, monitor activity, and ensure that role-based access control is enforced throughout the system. Sub Admins are specifically managed by the Super Admin, allowing hierarchical control where Sub Admins can perform delegated administrative tasks such as registering drivers and agents, managing QR codes, and reviewing requests.

Yegna Taxi

Dashboard

Driver Payments

Agent Commissions

Analytics

User Management

Sub-admins

Fermata Management

Admin User

SUPER ADMIN

Logout

User Management

Search by name, phone, or user ID...

All Roles

NAME	USER ID	PHONE	ROLE	BALANCE	REGISTERED	ACTIONS
radd	P1769191002662413	0985246237	Passenger	0.00 ETB	23/01/2026	Details
dsaaaa	P1769190914470403	0985246235	Passenger	440.00 ETB	23/01/2026	Details
Betremariam Yihenew	D1769190644500673	0985246234	Driver	75.00 ETB	23/01/2026	Details
demeke	D1769158734449760	0923393685	Driver	0.00 ETB	23/01/2026	Details

Yegna Taxi

Dashboard

Driver Payments

Agent Commissions

Analytics

User Management

Sub-admins

Fermata Management

Admin User

SUPER ADMIN

Logout

Sub-admins

Sub-admin Management

Create Sub-admin

NAME	USER ID	PHONE	EMAIL	CREATED AT	ACTIONS
Sub Admin User	SA1764989987872599	0987654321	subadmin@example.com	06/12/2025	

Figure 33 Web Admin - User Management and Sub-admin Management

Fare Management Implementation (Admin Side)

The fare management module allows the Super Admin to **define, update, and assign fares** to different taxis or routes. This ensures that all drivers and passengers are charged accurately according to city transport rules.

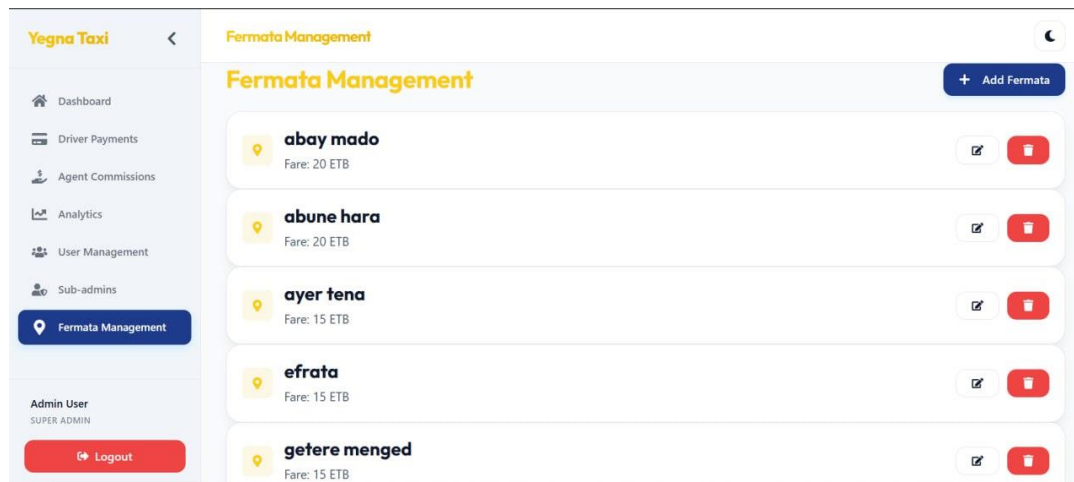


Figure 34 Web Admin - Fermata (Fare) Management

3.4 Access Control and Security

Overview

Access control is a critical component of the Tap Fare shared transport payment system, ensuring that only authorized users can access specific features and sensitive information. The system manages five distinct user roles, each with carefully defined permissions to protect user profiles, transaction data, payment information, QR codes, and administrative functions. This comprehensive security framework implements multiple layers of protection, from authentication and authorization to encryption and secure payment processing.

User Authentication

The Tap Fare system enforces robust authentication mechanisms to verify user identities before granting access to any sensitive features or data:

Authentication Methods

JWT Token-Based Authentication: The system uses JSON Web Tokens (JWT) for stateless authentication. Upon successful login, users receive a cryptographically signed token that must be included in the Authorization header for all subsequent requests.

Phone and Email Verification: Users authenticate using their registered phone number or email address combined with a secure password. The system validates credentials against hashed passwords stored in the database.

OTP (One-Time Password) Verification: For enhanced security, the system implements OTP verification during registration and critical account operations. OTPs are sent via email and have a limited validity period to prevent unauthorized access.

First Login Security: The system tracks first-time logins and can enforce additional security measures, such as mandatory password changes or profile completion, to ensure account security from the outset.

Token Management

- **Token Expiration:** JWT tokens have a limited lifespan to minimize the risk of token theft
- **Token Validation:** Every protected endpoint validates the token's signature, expiration, and associated user
- **Automatic Logout:** Sessions expire after a period of inactivity to prevent unauthorized access

Role-Based Access Control (RBAC)

The Integrated Digital Payment and Fare Management system implements a comprehensive Role-Based Access Control (RBAC) model with five distinct user roles. Each role has specific permissions tailored to their responsibilities within the ecosystem:

User Roles

1. SUPER_ADMIN

The highest privilege level in the system with complete administrative control.

Responsibilities:

- System-wide oversight and management
- Driver payment processing and management
- Agent commission management
- Analytics and reporting access
- User management (can manage all user types)
- QR code system management
- Complete transaction visibility

Key Privileges:

- Can create SUB_ADMIN accounts

- Can process and update driver payments
- Can manage agent commissions
- Access to comprehensive analytics dashboard
- Can view all users and transactions across the system
- Can assign and manage QR codes
- Can make payment to the agents and to the drivers
- Complete system configuration authority

2. SUB_ADMIN

Administrative role with delegated responsibilities for day-to-day operations.

Responsibilities:

- Driver and agent registration
- User management within assigned scope
- QR code management and distribution
- Transaction monitoring
- Limited analytics access

Key Privileges:

- Can register new drivers and agents
- Can view and manage users
- Can create and assign QR codes to agents
- Can view QR re-issue requests from agents
- Can approve QR code assignments
- Access to system analytics
- Cannot process payments or manage commissions (reserved for SUPER_ADMIN)

3. AGENT

Field agents who facilitate passenger onboarding and wallet top-ups.

Responsibilities:

- Passenger registration and onboarding
- Wallet top-up facilitation for passengers

- QR code distribution to passengers
- Commission tracking

Key Privileges:

- Can register new passengers
- Can initiate passenger wallet top-ups through Chapa payment gateway
- Can request QR code re-issuance for passengers
- Can view assigned passengers and their QR codes
- Can view own commission balance
- Can view transaction history related to their operations

Restrictions:

- Cannot access administrative functions
- Cannot view other agents' data
- Cannot process fare payments (driver function)
- Cannot modify system settings

4. DRIVER

Transport providers who collect fares from passengers.

Responsibilities:

- Scanning passenger QR codes
- Processing fare payments
- Processing refunds
- Managing bank account details for earnings
- Tracking earnings

Key Privileges:

- Can scan passenger QR codes to identify passengers
- Can process fare payments from passengers
- Can process refunds to passengers
- Can update own bank account details for payment disbursement
- Can view passenger information via QR lookup

- Can view own earnings and transaction history

Restrictions:

- Cannot register new users
- Cannot access administrative functions
- Cannot top up passenger wallets
- Cannot view other drivers' data

5. PASSENGER

End users who use the transport service.

Responsibilities:

- Managing personal profile
- Viewing wallet balance
- Viewing transaction history
- Managing payment methods
- Requesting QR code re-issuance

Key Privileges:

- Can view own profile and QR code
- Can deposit money into wallet via Chapa payment gateway
- Can view own wallet balance and transaction history
- Can request QR code re-issuance (through agent)
- Can change password and update profile
- Can view own payment history

Restrictions:

- Cannot access any other user's data
- Cannot perform administrative actions
- Cannot register other users
- Cannot process payments or top-ups
- Limited to self-service functions only

Access Restrictions and Authorization

The system implements middleware-based authorization that enforces access restrictions at the route level. Each endpoint specifies which roles are permitted to access it:

Authorization Middleware

The `authorize(...roles)` middleware function restricts access to specific routes based on user roles. When a request is made:

1. The system first authenticates the user via JWT token validation
2. The authorization middleware checks if the user's role is included in the allowed roles
3. If authorized, the request proceeds; otherwise, a 403 Forbidden response is returned

Endpoint Protection Strategy

- **Public Endpoints:** Login, registration, OTP verification, password reset (no authentication required)
- **Authenticated Endpoints:** Require valid JWT token but no specific role (e.g., view own profile, get balance)
- **Role-Restricted Endpoints:** Require both authentication and specific role authorization

Session Management

To prevent prolonged or unauthorized access, the Integrated Digital Payment and Fare Management system implements comprehensive session management:

Session Security Features

Token Expiration: JWT tokens contain an expiration timestamp. After expiration, users must re-authenticate to obtain a new token.

Inactivity Timeout: The system can detect and invalidate sessions after a period of inactivity (implementation via token refresh mechanisms).

Single Device Limitation (Optional): The system can be configured to invalidate previous tokens when a user logs in from a new device, preventing concurrent sessions.

Forced Logout: Administrators can force logout of specific users by invalidating their tokens in critical security situations.

First Login Flag: The system tracks first logins to enforce security policies like mandatory password changes or profile completion.

Security Measures

The Integrated Digital Payment and Fare Management system incorporates multiple layers of security to protect user data and ensure system integrity:

Data Protection

Password Security

- **Bcrypt Hashing:** All passwords are hashed using bcrypt with a high salt factor (typically 10+ rounds) before storage
- **No Plain Text Storage:** Passwords are never stored in plain text
- **Password Validation:** Enforces minimum password requirements (length, complexity)

Data Encryption

- **In-Transit Encryption:** All data transmitted between client and server is encrypted using HTTPS/TLS protocols
- **Sensitive Data Encryption:** Payment information and other sensitive data are encrypted during transmission
- **Database Encryption:** Sensitive fields can be encrypted at rest in the database

QR Code Security

- **Unique QR Codes:** Each passenger receives a unique, cryptographically generated QR code
- **QR Code Validation:** Drivers can only scan valid, active QR codes assigned to passengers
- **Re-issuance Control:** QR code re-issuance requires agent involvement and admin approval to prevent abuse

Payment Security

Chapa Payment Gateway Integration

- **Secure Payment Processing:** All payment transactions are processed through Chapa, a PCI-DSS compliant payment gateway
- **Payment Verification:** The system verifies payment callbacks from Chapa using secure webhooks and transaction references
- **Transaction Tracking:** Every payment is tracked with unique transaction references for auditing and reconciliation

Transaction Integrity

- **Atomic Transactions:** Database operations use transactions to ensure data consistency
- **Balance Validation:** The system validates user balances before processing payments
- **Double-Spend Prevention:** Transaction locking mechanisms prevent duplicate charges

Payment Workflow Security

1. **Initialization:** Secure payment initialization with encrypted data
2. **Redirect:** Users are redirected to Chapa's secure payment page
3. **Verification:** Payment status is verified via secure callback from Chapa
4. **Confirmation:** Only verified payments update user balances

Network Security

Firewall Protection: The backend server is protected by firewalls that filter malicious traffic and block unauthorized access attempts.

Rate Limiting: API rate limiting prevents abuse, DDoS attacks, and brute force attempts on authentication endpoints.

CORS Configuration: Cross-Origin Resource Sharing (CORS) is configured to allow only authorized domains to access the API.

Input Validation: All user inputs are validated and sanitized to prevent injection attacks (SQL injection, XSS, etc.).

Audit and Monitoring

Transaction Logging: All transactions are logged with timestamps, user IDs, and related user information for audit trails.

Activity Monitoring: The system logs critical activities such as:

- User authentication attempts
- Role-based access violations
- Payment processing events
- QR code issuance and re-issuance
- Administrative actions

Error Handling: Secure error handling ensures that sensitive system information is not leaked through error messages.

Comprehensive Access Control Matrix

The following table outlines the detailed access control policies for all major system functionalities across the five user roles:

Action/Functionality	SUPER_ADMIN	SUB_ADMIN	AGENT	DRIVER	PASSENGER
Authentication & Profile					
Login()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
Register()	✗ Deny	✗ Deny	✗ Deny	✗ Deny	✓ Allow
VerifyOTP()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
ViewOwnProfile()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
EditOwnProfile()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
ChangePassword()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
ForgotPassword()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow

ResetPassword()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
User Management					
RegisterDriver()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
RegisterAgent()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
RegisterPassenger()	X Deny	X Deny	✓ Allow	X Deny	X Deny
ViewAllUsers()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
ViewUserById()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
ViewAssignedUsers()	✓ Allow	✓ Allow	✓ Allow*	X Deny	X Deny
CreateSubAdmin()	✓ Allow	X Deny	X Deny	X Deny	X Deny
Wallet & Balance					
ViewOwnBalance()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
InitializeDeposit()	X Deny	X Deny	X Deny	X Deny	✓ Allow
VerifyDeposit()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
Transactions					
TopUpPassengerWallet()	X Deny	X Deny	✓ Allow	X Deny	X Deny
ProcessFarePayment()	X Deny	X Deny	X Deny	✓ Allow	X Deny
ProcessRefund()	X Deny	X Deny	X Deny	✓ Allow	X Deny
ViewOwnTransactionHistory()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
ViewAllTransactions()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
ViewUserTransactions()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
QR Code Management					
ViewOwnQRCode()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
CreateQRCodeForPrinting()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
AssignQRCodeToPassenger()	✓ Allow	✓ Allow	X Deny	X Deny	X Deny
ScanQRCode()	X Deny	X Deny	X Deny	✓ Allow	X Deny

RequestQRReissue()	✗ Deny	✗ Deny	✓ Allow* *	✗ Deny	✓ Allow**
ViewQRReissueRequests()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
ApproveQRAssignment()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
ViewQRCodesForAgent()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
LookupPassengerByQR()	✗ Deny	✗ Deny	✗ Deny	✓ Allow	✗ Deny
Payment Management					
ViewDriverPayments()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
UpdateDriverPayments()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
DownloadDriverPaymentsExcel()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
UpdateDriverBankDetails()	✗ Deny	✗ Deny	✗ Deny	✓ Allow	✗ Deny
ViewDriverEarnings()	✓ Allow	✗ Deny	✗ Deny	✓ Allow*	✗ Deny
ViewAgentCommissions()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
UpdateAgentCommissions()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
DownloadAgentCommissionsExcel()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
ViewOwnCommissionBalance()	✗ Deny	✗ Deny	✓ Allow	✗ Deny	✗ Deny
Analytics & Reporting					
ViewSystemAnalytics()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
ViewUserStatistics()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
ViewTransactionTrends()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
GenerateReports()	✓ Allow	✓ Allow	✗ Deny	✗ Deny	✗ Deny
DownloadExcelReports()	✓ Allow	✗ Deny	✗ Deny	✗ Deny	✗ Deny
File Management					
UploadProfilePhoto()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow
ViewProfilePhoto()	✓ Allow	✓ Allow	✓ Allow	✓ Allow	✓ Allow

Table 12 Access Control Matrix

Legend:

- **✓ Allow:** User role has permission to perform this action
- **✗ Deny:** User role does not have permission to perform this action
- Restricted to own data only (e.g., drivers can only view their own earnings, agents can only view their assigned users)
- Workflow-based permission (e.g., passengers can request QR re-issuance, but agents make the actual request to admins)

Additional Security Considerations**Data Privacy and Compliance**

Personal Data Protection: The system collects and stores only necessary personal information. Access to personal data is restricted based on role and necessity.

Data Retention: Transaction and audit logs are retained for a defined period for compliance and dispute resolution.

Right to Access: Users can view their own data through the profile and transaction history endpoints.

Secure Development Practices

Input Validation: All user inputs undergo validation using dedicated validator middleware to prevent injection attacks and malformed data.

Parameterized Queries: Database queries use Prisma ORM with parameterized statements to prevent SQL injection.

Error Handling: Generic error messages are returned to clients, while detailed errors are logged server-side to prevent information leakage.

Dependency Management: Regular updates of dependencies to patch security vulnerabilities.

Incident Response

Security Monitoring: Continuous monitoring for suspicious activities, multiple failed login attempts, or unusual transaction patterns.

Account Lockout: Accounts can be temporarily locked after multiple failed authentication attempts.

Emergency Access Control: Ability to immediately revoke access for compromised accounts.

Audit Trail: Comprehensive logging of all critical operations for forensic analysis and incident investigation.

Conclusion

The Integrated Digital Payment and Fare Management shared transport payment system implements a robust, multi-layered security and access control framework. Through the combination of JWT-based authentication, comprehensive role-based access control, secure payment processing via Chapa, and extensive security measures, the system ensures:

- Confidentiality: Sensitive data is protected from unauthorized access
- Integrity: Data cannot be modified by unauthorized parties
- Availability: Legitimate users can access the system when needed
- Accountability: All actions are logged for audit purposes
- Non-repudiation: Transaction records provide proof of actions taken

This security architecture provides a solid foundation for a trusted, reliable shared transport payment system while maintaining the flexibility to adapt to evolving security threats and compliance requirement

Chapter Four: Implementation

This section describes how the Integrated Digital payment system for shared taxi was implemented based on the detailed design specifications. It explains the system structure, key functional implementations, database realization, and user interface development. Selected screenshots of the implementation are included to demonstrate the correctness and completeness of the developed system.

System Architecture and Structure

The Integrated Digital payment system for shared taxi system was implemented using a layered architecture to ensure modularity, scalability, and maintainability. The system is logically divided into controllers, services, and data models.

Controllers handle incoming API requests and responses, services contain the core business logic such as payment processing and QR validation, and data models manage interaction with the database. This separation of concerns simplifies debugging and allows future enhancements without affecting unrelated components.

Below we have attached the image of the folder structure of the system for all the three systems we implemented. The web interface for the admin the mobile app and the backend structure are include here.

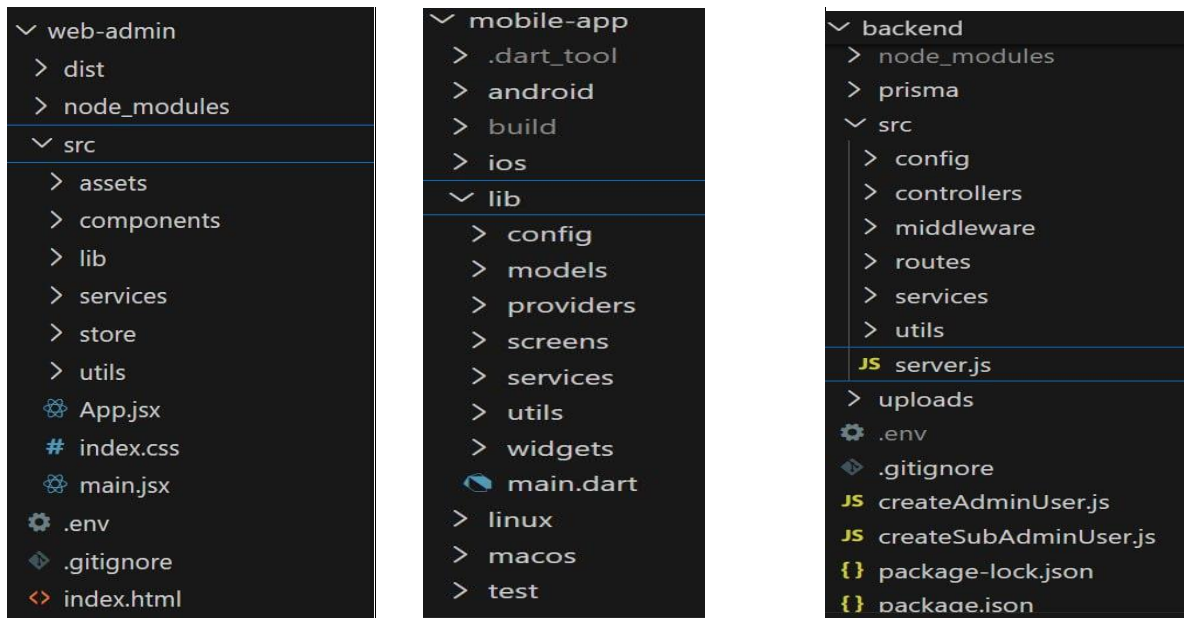


Figure 35 System Folder Structure

Authentication and Authorization Implementation

Authentication was implemented using secure login mechanisms with token-based authentication. Users are required to log in using their registered credentials, after which a secure token is generated and used for subsequent requests.

Role-based access control was enforced throughout the system to ensure that passengers, drivers, agents, and administrators can only access functionalities permitted to their roles. Middleware logic was used to protect sensitive endpoints and prevent unauthorized access.


```

const { verifyToken } = require('../utils/jwt');
const prisma = require('../config/database');

const authenticate = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];

    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Authentication required',
      });
    }

    const decoded = verifyToken(token);

    if (!decoded) {
      return res.status(401).json({
        success: false,
        message: 'Invalid or expired token',
      });
    }
  }
}

```

Figure 36 Authentication and Role-check Middleware Code

Transaction and Payment Processing Implementation

Core transaction workflows were implemented to support passenger top-ups, QR-based fare payments, refunds, and commission calculations. Each financial operation is handled atomically to ensure data consistency and prevent partial updates.

Agent commissions are automatically calculated during passenger top-ups, while driver earnings are updated in real time during fare payments. All transactions are recorded and linked to the appropriate users for tracking and reporting purposes.

```

const processFarePayment = async (req, res, next) => {
  const { qrCode } = req.body;
  let { amount } = req.body;
  const driverId = req.user.id;

  // Get driver to check for assigned Fermata
  const driver = await prisma.user.findUnique({
    where: { id: driverId },
    include: { fermata: true },
  });

  if (!driver || driver.role !== "DRIVER") {
    return res.status(403).json({
      success: false,
      message: "Only drivers can process fare payments",
    });
  }

  // Use tariff if fermata is assigned, otherwise use provided amount
  if (driver.fermata) {
    amount = driver.fermata.fare;
  } else if (!amount) {
    return res.status(400).json({
      success: false,
      message: "No tariff assigned to driver and no amount provided",
    });
  }
}

```

```

await prisma.$transaction(async (tx) => {
  // Deduct from passenger balance
  await tx.balance.update({
    where: { userId: passenger.id },
    data: {
      currentBalance: {
        decrement: amount,
      },
    },
  });

  // Add to driver earnings
  await tx.driverEarnings.upsert({
    where: { driverId },
    create: {
      driverId,
      totalEarnings: amount,
    },
    update: {
      totalEarnings: {
        increment: amount,
      },
    },
  });

  // Create transactions
}

```

```

// Get updated driver earnings
const driverEarnings = await prisma.driverEarnings.findUnique({
  where: { driverId },
});

res.json({
  success: true,
  message: "Fare payment processed successfully",
  data: {
    passenger: {
      name: passenger.name,
      userId: passenger.userId,
    },
    amount: amount.toString(),
    driverTotalEarnings: driverEarnings.totalEarnings.toString(),
  },
});
} catch (error) {
  next(error);
}

```

Figure 37 Transaction and Payment Processing Implementation Code

QR Code Management Implementation

QR code functionality is a central feature of the Integrated Digital Payment and Fare Management system. QR codes are generated for passengers and validated during fare collection by drivers. The system also supports printed QR codes for users without smartphones, which can be assigned and managed by agents or administrators.

Validation logic ensures that only active and valid QR codes can be used for payments, reducing the risk of misuse or fraud.

```

class _QRScannerState extends ConsumerState<QRScanner> {
  Future<void> _requestCameraPermission() async {
    // Check if permission is already granted
    PermissionStatus status = await Permission.camera.status;

    if (status.isGranted) {
      // Permission already granted, start the scanner
      _startScanner();
    } else {
      // Request permission
      status = await Permission.camera.request();
      if (status.isGranted) {
        _startScanner();
      } else {
        // Show dialog to explain why we need camera permission
        if (mounted) {
          setState(() {
            errorMessage = 'Camera permission is required to scan QR codes';
          });
        }
      }
    }
  }

  Future<void> _startScanner() async {
    if (mounted) {
      try {

```

```

const passenger = await prisma.user.findUnique({
  where: {
    id: true,
    name: true,
    email: true,
    phone: true,
    role: true,
    userId: true,
    photoUrl: true,
  },
});

if (!passenger) {
  return res.status(404).json({
    success: false,
    message: "Passenger not found",
  });
}

if (passenger.role !== "PASSENGER") {
  return res.status(400).json({
    success: false,
    message: "Scanned QR code does not belong to a passenger",
  });
}

```

Figure 38 QR Code Management Implementation

Passenger Wallet Initialization via Chapa

Chapa payment gateway was integrated to support wallet initialization and balance top-up for passengers. Passengers with smartphones can top up their own wallets directly through the integrated Chapa payment system, and their wallet balances are updated automatically upon successful payment verification.

In addition, agents are authorized to top up the wallets of passengers who do not own smartphones. This ensures inclusivity and allows all users to access the digital payment system regardless of device availability.

The backend implementation of the Chapa payment integration is shown in the image below.

```
const randomLetter = Math.random().toString(36).substring(2, 10);
const tx_ref = randomLetter + "_" + userId;

const callbackUrl = `${
  process.env.BACKEND_URL || "http://localhost:3000"
}/api/balance/deposit/verify`;

const returnUrl = `${callbackUrl}?tx_ref=${encodeURIComponent(tx_ref)}`;

const payment = await initializeChapaPayment(
  amount,
  user.email || `${user.phone}@YegnaTaxi.com`,
  user.phone,
  user.name.split(" ")[0] || user.name,
  user.name.split(" ").slice(1).join(" ") || "",
  tx_ref,
  callbackUrl,
  returnUrl
);

res.json({
  success: true,
  message: "Payment initialized",
  data: {
    checkoutUrl: payment.checkoutUrl
  }
});
```

```

const randomLetter = Math.random().toString(36).substring(2, 10);
const tx_ref = randomLetter + "_" + userId;

const callbackUrl = `${
  process.env.BACKEND_URL || "http://localhost:3000"
}/api/balance/deposit/verify`;

const returnUrl = `${callbackUrl}?tx_ref=${encodeURIComponent(tx_ref)}`;

const payment = await initializeChapaPayment(
  amount,
  user.email || `${user.phone}@YegnaTaxi.com`,
  user.phone,
  user.name.split(" ")[0] || user.name,
  user.name.split(" ").slice(1).join(" ") || "",
  tx_ref,
  callbackUrl,
  returnUrl
);

res.json({
  success: true,
  message: "Payment initialized",
  data: {
    checkoutUrl: payment.checkoutUrl
  }
});

```

Figure 39 Chapa Payment Integration Backend Implementation

Summary

The Integrated Digital Payment and Fare Management system was successfully implemented according to the proposed design. All major features, including authentication, QR-based payments, transaction handling, and role-based access control, were fully realized. The modular structure and secure implementation provide a solid foundation for deployment and future enhancements.

Chapter-Five: Testing and Evaluation

Testing Methodology

Testing was conducted to verify that the Integrated Digital Payment and Fare Management system meets its functional and integration requirements. The system was tested across three main components:

- Backend API
- Web-based Admin Portal
- Mobile Application (Flutter)

Testing focused on authentication, user management, QR code operations, transaction processing, and payment integration. Manual testing was primarily used, supported by API testing tools and real device testing for the mobile application.

Backend API Testing

Backend functionality was tested using Postman to ensure that all API endpoints behaved as expected.

Authentication and User Management

- User registration and login for all roles (passenger, driver, agent, admin)
- JWT-based authentication and token validation
- Mandatory password change for drivers and agents on first login
- Enforcement of role-based access control

Transaction Management

- Agent wallet top-up to passengers with commission calculation
- Driver fare collection using QR-based transactions
- Refund processing
- Transaction history retrieval
- Payment gateway integration (payment initialization, verification, and callback handling)

QR Code Operations

- QR code generation for passengers
- Creation of printed QR codes for non-smartphone users
- Assignment of QR codes to passengers
- QR code validation during scanning

Administrative APIs

- Driver earnings retrieval and payment status updates
- Agent commission tracking and payout processing
- Analytics data retrieval
- User listing with role-based filtering

Result: All tested API endpoints responded correctly and performed as expected.

Web Admin Portal Testing

The web-based admin portal was tested through a browser to validate administrative workflows and system management features.

Super Admin Features

- Dashboard analytics and system statistics
- Driver payment management (viewing earnings, exporting reports, updating payment status)
- Agent commission management and payout processing
- User management across all system roles

Sub-Admin Features

- Registration of drivers and agents

- Creation of printed QR codes for non-smartphone users
- Management of QR re-issue requests
- Tracking of QR code distribution

Result: All administrative features functioned correctly and supported the intended workflows.

Mobile Application Testing

The Flutter-based mobile application was tested to verify core user interactions for passengers, agents, and drivers.

Passenger Features

- User registration with automatic QR code generation
- Wallet balance display
- Account top-up using the payment gateway
- QR code display for fare payment
- Transaction history and profile management

Agent Features

- Dual wallet system (operating balance and commission balance)
- Registration of non-smartphone passengers
- Passenger wallet top-up with commission calculation
- Agent wallet top-up through the payment gateway

Driver Features

- QR code scanning using the device camera
- Fare collection workflow
- Refund handling
- Earnings and transaction history tracking

Result: The mobile application functioned correctly for all user roles and core workflows.

Integration and Security Testing

Integration testing was conducted to ensure smooth communication between system components.

- End-to-end workflows were tested, including registration, payment, and settlement
- Data synchronization between the backend, web portal, and mobile application was verified
- Secure password handling using hashing mechanisms was confirmed
- Transaction consistency and balance updates were validated
- Payment gateway flow was tested from initialization to balance update

Test Summary

Test Category	Number of Tests	Result
Backend API	35	Passed
Web Admin Portal	15	Passed
Mobile Application	28	Passed
Integration Testing	12	Passed
Total	90	Passed

Table 13 Test Summary

Testing results indicate that the Integrated Digital Payment and Fare Management system meets its functional requirements across all platforms. The system demonstrated reliable performance in authentication, transaction handling, QR-based payments, and administrative operations.

The integration between the backend, web portal, mobile application, and payment gateway functioned correctly, ensuring consistent data flow and secure transaction processing.

Based on the conducted tests, the system is considered stable and suitable for deployment in a real-world environment, subject to future enhancements and scalability testing.

References

- [1] Pressman, R. S. (2020). *Software Engineering: A Practitioner's Approach* (10th ed.). McGraw-Hill Education. (Requirements Engineering Chapter)
- [2] van Lamsweerde, A. (2009). *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Wiley.
- [3] IEEE Std 830-1998. (1998). *Recommended Practice for Software Requirements Specifications*. IEEE Computer Society.
- [4] Casciaro, M., & Mammino, L. (2020). *Node.js Design Patterns* (3rd ed.). Packt Publishing. (API and File Upload Best Practices)
- [5] Bahir Dar University, Bahir Dar Institute of Technology, Faculty of Computing. Annex_I_Documentation_template_For_Requirement_Analysis_DocumentRAD.pdf. (Template used for document structure).
- [6] Node.js Foundation, *Node.js Documentation*, Node.js, Available: <https://nodejs.org/en/docs>
- [7] Express.js, *Express – Node.js Web Framework*, Available: <https://expressjs.com>
- [8] Flutter Team, *Flutter Documentation*, Google, Available: <https://docs.flutter.dev>
- [9] PostgreSQL Global Development Group, *PostgreSQL Documentation*, Available: <https://www.postgresql.org/docs>
- [10] MongoDB Inc., *MongoDB Manual*, Available: <https://www.mongodb.com/docs>
- [11] Chapa Financial Technologies, *Chapa Payment Gateway API Documentation*, Available: <https://developer.chapa.co>
- [12] JSON Web Token, *JWT Introduction*, Available: <https://jwt.io/introduction>
- [13] Niels Provos and David Mazières, *A Future-Adaptable Password Scheme*, bcrypt, Available: <https://en.wikipedia.org/wiki/Bcrypt>
- [14] PlantUML, *PlantUML Language Reference Guide*, Available: <https://plantuml.com>

Appendix:

Appendix A: Observation Notes from Minibus Terminal

Date: December 18, 2025

Location: Bahir Dar Main Minibus Terminal

Time: 6:30-8:30 AM Peak Hour

Observer: Group Members

Time	Activity Observed	Duration	Issue Noted
2:30(LT)	15 passengers boarding, cash collection	8 min	Driver arguing with passenger over change
7:15(LT)	Fare collection during motion	10 min	Passenger dropped ETB 50 note
8:30(LT)	Return trip - cash counting	12 min	Driver short ETB 120 from expected

Table 14 Field Observation Notes

Key Findings:

- Average boarding delay: minutes lost due to cash handling
- Disputes observed: 4 incidents in 2 hours
- Cash loss incidents: 1 dropped notes totaling ETB 40
- Peak hour chaos: Drivers handling multiple transactions simultaneously



Appendix B: Chapa vs Telebirr Competitor Analysis

D.1 Feature Comparison: Chapa vs Telebirr for Transport Payments

Criteria	Chapa	Telebirr	Gap Addressed by Proposed System
API Response Time	1.2s avg	2.1s avg	<2s QR deduction (NFR-P-01)
Driver Dashboard	None	Basic merchant dashboard	Real-time earnings (F_DRIV_4)
Bulk Payouts	Manual	Automated (min 100 users)	Manual + tracking (BR-01)
Offline Support	No	Partial (USSD)	Planned Caching
Ethiopia Coverage	90% urban	95% nation wide	Urban focus
Agent Commissions	None	None	3% auto-calc (F_AGNT_5)
Admin Analytics	Basic reports	Call center only	Full dashboards (F_REP_1)

Table 15 Chapa vs Telebirr

D.2 Performance Metrics (2025 Data)

Transaction Volume:

- Chapa: 2.5M transactions/month (retail dominant)
- Telebirr: 45M transactions/month (telecom/airtime dominant)
- Transport Sector: <2% of total volume → ETB 1.2B annual opportunity

Success Rates:

- Chapa: 98.2% first-time success
- Telebirr: 97.8% first-time success
- Peak Hour Failure: 12-15% (network congestion)

D.4 Market Positioning

Chapa Strengths: Fastest API, developer-friendly, urban merchant adoption

Chapa Weaknesses: No transport workflows, no driver tools

Telebirr Strengths: Massive scale, nationwide coverage

Telebirr Weaknesses: Slow API, no QR, telecom-focused

Proposed System Differentiation:

1. Transport-First: Built for minibus fare collection (not retrofitted)
2. Hybrid Access: 100% coverage vs 45% smartphone limitation
3. Agent Economy: Creates revenue for many agents
4. Bureau Partnership: Exclusive dashboards for regulators